



Performance Modeling of Computer Systems and Networks

Progetto: Modellazione, simulazione e valutazione delle prestazioni del workflow di un'applicazione e-commerce

Simone Guidato

Matricola: 0354367

simone.guidato@students.uniroma2.eu

Anno Accademico 2025/2026

Indice

1	Descrizione del sistema	4
1.1	Contesto	4
1.2	Il sistema analizzato	4
1.2.1	Assunzioni di modellazione e modifiche introdotte	4
2	Problematiche del sistema	5
3	Obiettivi dello studio	6
4	Modello Concettuale e Dinamiche di Sistema	6
4.1	Architettura Logica e Routing (Class-Switching)	7
4.2	Rappresentazione dello Stato del Sistema	8
4.3	Disciplina di Servizio: Processor Sharing Puro	8
4.4	Eventi Discreti e Transizioni di Stato	8
4.5	Assunzioni Operative e Limiti del Modello	9
5	Modello delle Specifiche	9
5.1	Processo di Arrivo, Visite e Distribuzioni di Servizio	9
5.2	Scenari Operativi e Tempi Medi di Servizio	11
5.3	Domande di Servizio Aggregate e Throughput Bound	11
5.4	Scenario di Heavy Workload	13
6	Modello Computazionale	13
6.1	Processor Sharing: Approccio Tag-Based	13
6.2	Esempio Numerico di Funzionamento	14
6.3	Determinazione del Prossimo Completamento	15
6.4	Risoluzione e Avanzamento del Ciclo di Simulazione	16
6.5	Raccolta delle Metriche	17
6.6	Criteri di Terminazione	18
7	Verifica	18
7.1	Test 1: Coerenza Interna e Legge di Little	18

7.2	Test 2: Verifica Controllata del Processor Sharing M/G/1	20
7.3	Test 3: Verifica del Generatore Pseudo-Casuale	21
7.3.1	Verifica delle Distribuzioni di Servizio	22
8	Validazione del Modello	22
8.1	Protocollo Sperimentale dello Scan di Carico	23
8.2	Conservazione dei Flussi e Throughput	23
8.3	Validazione delle Curve Globali di Risposta	24
8.4	Errore Relativo rispetto alla Soluzione Analitica	26
8.5	Validazione delle Metriche di Stazione	27
8.6	Legge di Little lungo lo Scan	28
8.7	Confronto con il Caso di Studio e Interpretazione del Punto Critico	29
9	Calibrazione Statistica e Analisi del Transitorio	29
9.1	Scelta del Punto Operativo Prossimo alla Saturazione	30
9.2	Analisi del Transitorio e Dimensionamento del Warm-Up	30
9.2.1	Visualizzazione della Convergenza Multi-Realizzazione	31
9.3	Autocorrelazione e Scelta della Dimensione dei Batch	34
10	Analisi a Regime: Confronto 1FA vs 2FA	37
10.1	Configurazione dell'Esperimento e Gestione dell'RNG	37
10.2	Batching delle Metriche	38
10.3	Tempo di Risposta e Penalità della 2FA	38
10.4	Confronto Quantitativo al Punto Critico	40
10.5	Distribuzione della Congestione tra i Centri	41
10.6	Decomposizione del Tempo di Risposta mediante Job Tracing	42
11	Insensibilità e Variabilità del Servizio al Server B	43
11.1	Configurazione Sperimentale	43
11.2	Parametrizzazione della Distribuzione Iperesponenziale	44
11.3	Metriche Medie e Proprietà di Insensibilità	45
11.4	Effetto della Variabilità sui Tempi Individuali	46
12	Analisi in Condizioni di Sovraccarico (Heavy Load)	48

12.1 Configurazione Sperimentale a Orizzonte Finito	48
12.2 Dinamica di Accumulo della Popolazione	49
12.3 Confronto Esponenziale vs Iperesponenziale in Overload	50
12.4 Dimensionamento della Capacità	52
12.4.1 Scenario 1FA	52
12.4.2 Scenario 2FA	53
13 Conclusioni	54

1 Descrizione del sistema

1.1 Contesto

Le moderne applicazioni web implementano frequentemente la propria logica di business attraverso workflow composti da una sequenza di operazioni eseguite su più risorse di calcolo. La modellazione prestazionale di tali sistemi mediante reti di code risulta particolarmente interessante quando una stessa richiesta visita più volte la medesima risorsa, svolgendo operazioni differenti e richiedendo quindi differenti quantità di servizio.

Questo lavoro considera il workflow di sottomissione di un ordine in un'applicazione di e-commerce, nel quale ogni richiesta attraversa le risorse del sistema secondo un percorso deterministico multi-visita. L'obiettivo è costruire un modello a eventi discreti che permetta di studiare il comportamento del sistema al variare del carico, della configurazione di sicurezza dei pagamenti e della distribuzione dei tempi di servizio.

1.2 Il sistema analizzato

Il sistema analizzato è una versione semplificata del workflow di sottomissione di un ordine in un'applicazione di e-commerce, ispirata al caso di studio “*Simulation of the Workflow of a Web App*” [1].

L'infrastruttura è composta da tre centri di servizio:

- **Server A:** rappresenta il Front-End dell'applicazione e viene visitato tre volte durante l'elaborazione di ciascuna richiesta. Le tre visite corrispondono a operazioni differenti e sono pertanto caratterizzate da differenti *service demand*.
- **Server B:** rappresenta il componente Web Application/Database e viene visitato una volta da ciascuna richiesta. Il *service demand* medio della visita è pari a 0.8 s.
- **Server P:** rappresenta il Gateway di Pagamento ed è visitato una volta durante la fase di autorizzazione del pagamento.

Ogni richiesta (*job*) segue il percorso deterministico

$$A \rightarrow B \rightarrow A \rightarrow P \rightarrow A,$$

effettuando quindi tre visite al Server A e una visita rispettivamente ai Server B e P.

Per distinguere le differenti operazioni associate alle tre visite al Front-End, a ciascun job è associata una classe logica che viene aggiornata durante l'esecuzione del workflow. La classe corrente determina il successivo centro di servizio e il *service demand* associato alla visita.

1.2.1 Assunzioni di modellazione e modifiche introdotte

Rispetto al caso di studio di riferimento, il modello adotta le seguenti assunzioni:

- **Disciplina di servizio:** tutti i centri adottano una disciplina *Processor Sharing* (PS) pura, nella quale la capacità del server viene ripartita istantaneamente ed equamente tra tutti i job presenti.

- **Processo di arrivo:** gli arrivi esterni sono descritti mediante un processo di Poisson, con tempi di interarrivo esponenziali.
- **Sistema aperto:** il tasso di arrivo λ è imposto esogenamente e non viene modellato alcun *think time* degli utenti.
- **Capacità dei buffer:** le code sono considerate a capacità infinita; non si verificano quindi perdite di richieste per overflow.
- **Overhead aggiuntivi:** i tempi di routing tra i centri, di commutazione della classe e gli eventuali ritardi di comunicazione non vengono modellati esplicitamente.
- **Distribuzione del tempo di servizio del Server B:** nel caso di studio di riferimento i tempi di servizio sono esponenziali. Nel modello nominale del presente lavoro il Server B viene invece caratterizzato mediante una distribuzione **iperesponenziale a due rami** con coefficiente di variazione $CV = 2$, mantenendo invariato il valore medio $E[S_B] = 0.8$ s.

Tale modifica costituisce una scelta sperimentale finalizzata a studiare l'effetto di una maggiore variabilità del servizio. Nel dominio di stabilità, la disciplina M/G/1-PS presenta proprietà di insensibilità rispetto alla forma della distribuzione del tempo di servizio per le metriche medie stazionarie, mentre le caratteristiche di dispersione e di coda dei tempi di risposta possono risultare significativamente differenti.

2 Problematiche del sistema

La struttura multi-visita del workflow genera una distribuzione non uniforme del carico tra i tre centri di servizio. In particolare, ogni richiesta visita tre volte il Server A, una volta il Server B e una volta il Server P; il carico effettivo di ciascuna risorsa dipende quindi sia dal numero di visite sia dal relativo *service demand*.

Nello scenario 1FA, le tre visite al Server A producono una domanda aggregata pari a $D_A = 0.2 + 0.4 + 0.1 = 0.7$ s, mentre il Server B presenta $D_B = 0.8$ s. Poiché D_B costituisce la maggiore domanda di servizio dell'intera rete, il Server B è il candidato naturale a rappresentare il **collo di bottiglia** del sistema. La sua capacità teorica massima è pertanto attesa nell'intorno di

$$X_{\max} = \frac{1}{D_B} = 1.25 \text{ req/s},$$

valore che verrà verificato analiticamente e sperimentalmente nelle sezioni successive.

Una seconda problematica riguarda l'introduzione dell'autenticazione a due fattori. Lo scenario 2FA non modifica la domanda del Server B, ma incrementa il lavoro richiesto al Gateway di Pagamento e durante l'ultima visita al Front-End. È quindi necessario valutare quanto tale maggiore sicurezza degradi il tempo di risposta complessivo e come modifichi la distribuzione della congestione all'interno della rete. Un ulteriore aspetto riguarda la **variabilità del servizio**. Il modello nominale adotta sul Server B una distribuzione iperesponenziale con $CV = 2$, mantenendo lo stesso tempo medio di servizio del caso esponenziale. Ciò permette di verificare sperimentalmente se le metriche medie risultino coerenti con la proprietà di insensibilità del Processor Sharing e, contemporaneamente, di quantificare l'effetto della maggiore variabilità sulle code della distribuzione dei tempi di risposta.

Infine, il sistema deve essere valutato anche rispetto a un incremento del carico fino a $\lambda = 1.4$ req/s. Poiché tale valore supera il throughput bound nominale del Server B, è necessario studiare

il comportamento della rete fuori dal dominio di stabilità e valutare, mediante un'analisi di *capacity planning*, quali interventi sulle risorse siano necessari per rendere sostenibile il nuovo workload.

3 Obiettivi dello studio

Lo studio si propone di rispondere alle seguenti domande:

- **Prestazioni al variare del carico.** Come variano il tempo medio di risposta end-to-end R , le popolazioni medie N_A , N_B e N_P , le utilizzazioni U_A , U_B e U_P e i throughput X_A , X_B e X_P al crescere del tasso di arrivo esterno $\lambda \in [0.50, 1.20]$ req/s?
- **Impatto dell'autenticazione a due fattori.** Qual è il costo prestazionale dell'introduzione della 2FA rispetto allo scenario 1FA a parità di carico offerto? In particolare, come cambia il tempo di risposta end-to-end e quali centri assorbono il lavoro aggiuntivo introdotto dai maggiori *service demand* del Server P e della terza visita al Server A?
- **Identificazione del collo di bottiglia.** Quale centro determina il throughput massimo sostenibile dalla rete e come evolve la sua utilizzazione all'avvicinarsi della saturazione?
- **Effetto della variabilità del servizio.** A parità di tempo medio di servizio $E[S_B] = 0.8$ s, la sostituzione della distribuzione esponenziale con una distribuzione iperesponenziale a $CV = 2$ modifica le metriche medie a regime? Quale effetto produce invece sulla variabilità e sui percentili di coda dei tempi di permanenza al Server B?
- **Comportamento in condizioni di sovraccarico.** Il sistema è in grado di sostenere un incremento del workload fino a $\lambda = 1.4$ req/s? In caso contrario, come evolve nel tempo la popolazione del collo di bottiglia e quale ruolo assume la distribuzione del servizio nella dinamica transitoria fuori dal regime stazionario?
- **Dimensionamento della capacità.** Quale incremento della capacità delle risorse sarebbe necessario per sostenere il workload di picco? In particolare, quali effetti produce una riduzione della domanda del Server B da 0.8 s a 0.4 s e quale risorsa diventa il nuovo collo di bottiglia negli scenari 1FA e 2FA?

Per rispondere a tali domande viene seguito un processo sperimentale articolato in verifica del simulatore, validazione rispetto ai risultati analitici, analisi del transitorio, determinazione del warm-up, studio dell'autocorrelazione e scelta della dimensione dei batch.

Le stime a regime vengono successivamente ottenute mediante il metodo delle *Batch Means* e corredate da intervalli di confidenza al 95%. Le condizioni di sovraccarico, non ammettendo una distribuzione stazionaria, vengono invece analizzate mediante più realizzazioni a orizzonte temporale finito.

4 Modello Concettuale e Dinamiche di Sistema

L'infrastruttura in esame viene rappresentata mediante una rete di code aperta (*Open Queueing Network*). Le richieste arrivano dall'esterno secondo un processo stocastico, attraversano i centri

di servizio seguendo un routing deterministico multi-visita e, completata l'ultima operazione, abbandonano definitivamente il sistema.

4.1 Architettura Logica e Routing (Class-Switching)

Come illustrato in Figura 1, il ciclo di vita di ogni richiesta segue il percorso

$$A \rightarrow B \rightarrow A \rightarrow P \rightarrow A.$$

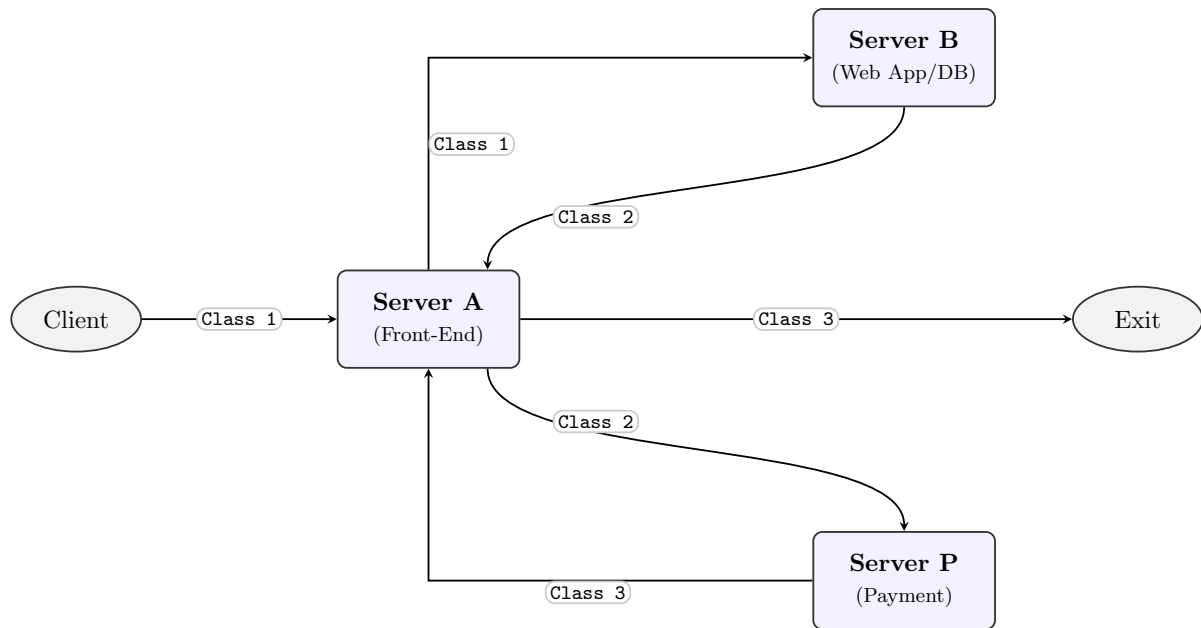


Figura 1: Modello concettuale del workflow: $A \rightarrow B \rightarrow A \rightarrow P \rightarrow A$. L'identificativo di classe permette di distinguere le differenti operazioni associate alle tre visite al Server A.

La natura multi-visita del workflow viene rappresentata mediante il concetto di *classe del job*. L'identificativo di classe viene aggiornato durante l'instradamento mediante transizioni logiche a ritardo nullo (*class switching*):

1. **Classe 1 – Autenticazione e Database.** La richiesta entra nel sistema e viene inizialmente servita dal Server A. Terminata la prima visita, viene inoltrata al Server B per le operazioni relative alla Web Application e al Database.
2. **Classe 2 – Checkout e Pagamento.** Dopo il completamento sul Server B, il job passa logicamente alla Classe 2, ritorna al Server A per la generazione dell'ordine e viene successivamente inoltrato al Server P per l'autorizzazione del pagamento.
3. **Classe 3 – Conferma.** Terminata l'elaborazione sul Server P, il job passa alla Classe 3, effettua l'ultima visita al Server A per la conferma dell'ordine e successivamente abbandona la rete.

Il cambio di classe non rappresenta una nuova richiesta e non introduce alcun ritardo temporale: serve esclusivamente a descrivere il progresso della stessa entità lungo il workflow e a selezionare il corretto *service demand* per la visita successiva.

4.2 Rappresentazione dello Stato del Sistema

Ai fini dell'analisi delle popolazioni, in ogni istante t lo stato di occupazione della rete può essere rappresentato mediante il vettore

$$\mathbf{N}(t) = [N_{A,1}(t), N_{A,2}(t), N_{A,3}(t), N_B(t), N_P(t)], \quad (1)$$

dove $N_{A,c}(t)$ indica il numero di job di classe c presenti presso il Server A, mentre $N_B(t)$ e $N_P(t)$ rappresentano rispettivamente le popolazioni dei Server B e P.

La popolazione complessiva del Front-End è quindi

$$N_A(t) = N_{A,1}(t) + N_{A,2}(t) + N_{A,3}(t). \quad (2)$$

Tale vettore rappresenta una *proiezione di occupazione* dello stato del sistema, sufficiente per descrivere le metriche di popolazione e utilizzazione analizzate nel seguito. Non costituisce, tuttavia, l'intero stato computazionale del simulatore: l'esecuzione richiede anche informazioni quali il prossimo arrivo, lo stato del tempo virtuale dei server e i tag di completamento dei job attivi.

Dal punto di vista implementativo, le tre classi presenti sul Server A non corrispondono a tre code fisiche separate. Il centro mantiene un unico insieme di job attivi e la classe corrente è memorizzata come attributo del singolo job. La struttura dati utilizzata per implementare efficientemente tale insieme e il meccanismo dei tag virtuali saranno descritti nella Sez. 6.

4.3 Disciplina di Servizio: Processor Sharing Puro

Tutti i centri di servizio adottano una disciplina **Processor Sharing (PS) puro**. In tale disciplina non esiste una distinzione tra job in attesa e job in servizio: tutte le richieste presenti ricevono contemporaneamente una quota della capacità del centro.

Indicando con C_k la capacità del server k e con $N_k(t)$ il numero di job attivi, quando $N_k(t) > 0$ ciascun job riceve una quota istantanea di capacità pari a

$$r_k(t) = \frac{C_k}{N_k(t)}. \quad (3)$$

Nel simulatore viene adottata una capacità normalizzata $C_A = C_B = C_P = 1$. Di conseguenza, un nuovo arrivo presso un centro riduce immediatamente la quota di capacità assegnata ai job già presenti, mentre un completamento ridistribuisce la capacità liberata tra i job rimanenti. La congestione sotto Processor Sharing si manifesta attraverso la progressiva dilatazione del tempo di permanenza delle richieste contemporaneamente presenti.

4.4 Eventi Discreti e Transizioni di Stato

L'evoluzione temporale della rete è determinata principalmente da due categorie di eventi discreti:

- **Arrivo esterno (*Arrival*).** Viene generato un nuovo job di Classe 1, che entra nel Server A. Per la visita corrente viene campionata la relativa richiesta di servizio dalla distribuzione associata alla coppia server-classe.
- **Completamento (*Completion*).** Quando un job completa il servizio presso un centro, viene rimosso dall'insieme dei job attivi e viene registrato il tempo di permanenza relativo alla visita appena conclusa. Il **WorkflowRouter** determina quindi la transizione successiva: se il workflow non è terminato, aggiorna la classe del job, campiona il nuovo requisito di servizio e lo inserisce nel centro successivo; dopo il completamento della terza visita al Server A, il job abbandona definitivamente il sistema e viene registrato il tempo di risposta end-to-end.

Per ciascun job vengono inoltre mantenuti i tempi relativi alle singole visite, utilizzati successivamente per la decomposizione del tempo di risposta complessivo mediante il *job tracing*.

4.5 Assunzioni Operative e Limiti del Modello

Per isolare gli effetti del routing multi-visita e della disciplina Processor Sharing, vengono adottate le seguenti assunzioni:

- **Buffer a capacità infinita.** Nessuna richiesta viene scartata per overflow delle code. In condizioni di sovraccarico la popolazione può quindi crescere senza un limite imposto artificialmente dal modello.
- **Routing a ritardo nullo.** I cambi di classe e il trasferimento logico tra i centri non introducono tempi di servizio aggiuntivi.
- **Ritardi di rete non modellati esplicitamente.** Eventuali tempi di comunicazione con componenti esterni sono considerati trascurabili oppure inclusi nelle domande di servizio dei centri.
- **Capacità statica dei server.** Il modello nominale non prevede meccanismi automatici di scaling. Le capacità dei centri rimangono quindi costanti durante ciascun esperimento, permettendo di identificare direttamente il collo di bottiglia.

5 Modello delle Specifiche

5.1 Processo di Arrivo, Visite e Distribuzioni di Servizio

La traduzione dell'architettura logica descritta nella Sez. 4 in un modello prestazionale richiede la specifica dei processi di arrivo, del numero di visite effettuate presso ciascun centro e delle distribuzioni utilizzate per generare i tempi di servizio. Gli arrivi delle richieste dall'esterno sono descritti mediante un processo di Poisson di intensità λ . Di conseguenza, i tempi di interarrivo sono variabili aleatorie esponenziali con media

$$E[A] = \frac{1}{\lambda}. \quad (4)$$

Per le analisi condotte all'interno del dominio di stabilità, il tasso di arrivo esterno viene variato nell'intervallo

$$\lambda \in [0.50, 1.20] \text{ req/s}, \quad (5)$$

mentre il punto

$$\lambda = 1.40 \text{ req/s} \quad (6)$$

viene studiato separatamente come condizione di sovraccarico mediante simulazioni a orizzonte temporale finito.

Il routing deterministico

$$A \rightarrow B \rightarrow A \rightarrow P \rightarrow A$$

impone a ciascuna richiesta esterna il seguente numero di visite:

$$V_A = 3, \quad V_B = 1, \quad V_P = 1. \quad (7)$$

In assenza di perdite e in condizioni di conservazione del flusso, i corrispondenti throughput di visita risultano quindi

$$X_A = 3\lambda, \quad X_B = \lambda, \quad X_P = \lambda. \quad (8)$$

Ciascuna delle tre classi logiche attraversa il Server A una sola volta. Pertanto, il flusso associato a ciascuna classe presso il Front-End è pari a λ , mentre il flusso aggregato attraverso il Server A è pari a 3λ . Per quanto riguarda le distribuzioni del tempo di servizio, il modello adotta le seguenti specifiche:

- le tre visite al **Server A** sono caratterizzate da tempi di servizio esponenziali, la cui media dipende dalla classe corrente del job;
- il **Server P** adotta una distribuzione esponenziale del tempo di servizio;
- nel modello nominale il **Server B** adotta una distribuzione iperesponenziale a due rami (H_2), con coefficiente di variazione $CV = 2$ e tempo medio di servizio mantenuto pari a

$$E[S_B] = 0.8 \text{ s}. \quad (9)$$

Il Server B appartiene quindi, nel modello nominale, alla classe $M/G/1$ sotto disciplina Processor Sharing; più specificamente, la distribuzione utilizzata è una H_2 . La scelta iperesponenziale rappresenta una modifica sperimentale rispetto al caso di studio di riferimento, nel quale i tempi di servizio sono esponenziali. Per consentire sia la validazione rispetto al modello originale sia lo studio dell'effetto della variabilità, viene mantenuta anche una configurazione alternativa nella quale il Server B utilizza una distribuzione esponenziale con la stessa media $E[S_B] = 0.8 \text{ s}$. In questo modo, il confronto tra le due configurazioni modifica la forma e la variabilità della distribuzione di servizio, mantenendone invariato il valore medio.

5.2 Scenari Operativi e Tempi Medi di Servizio

Il workload viene parametrizzato mediante i tempi medi di servizio $E[S_{k,c}]$ associati alla visita di un job di classe c presso il centro k . Nel dominio di stabilità vengono considerate due configurazioni operative principali: 1FA e 2FA.

Scenario 1FA. Lo scenario **1FA** (*Single-Factor Authentication*) rappresenta la configurazione di base del sistema. I tempi medi di servizio per le singole visite sono riportati nella Tabella 1.

Tabella 1: Tempi medi di servizio per visita nello scenario 1FA.

Centro	Classe 1	Classe 2	Classe 3
Server A	0.20	0.40	0.10
Server B	0.80	–	–
Server P	–	0.40	–

In particolare, le tre visite al Front-End richiedono rispettivamente 0.20, 0.40 e 0.10 secondi medi di servizio.

Scenario 2FA. Lo scenario **2FA** (*Two-Factor Authentication*) rappresenta l'introduzione di un meccanismo di autenticazione più oneroso durante la fase di pagamento. Rispetto alla configurazione 1FA:

- il tempo medio della visita di Classe 2 al Server P aumenta da 0.40 s a 0.70 s;
- il tempo medio della visita di Classe 3 al Server A aumenta da 0.10 s a 0.15 s.

Le rimanenti richieste di servizio restano invariate.

Tabella 2: Tempi medi di servizio per visita nello scenario 2FA.

Centro	Classe 1	Classe 2	Classe 3
Server A	0.20	0.40	0.15
Server B	0.80	–	–
Server P	–	0.70	–

5.3 Domande di Servizio Aggregate e Throughput Bound

Ai fini dell'analisi della capacità è utile distinguere il tempo medio di servizio della *singola visita* dalla **domanda di servizio aggregata per richiesta esterna** presso ciascun centro. Poiché il routing è deterministico, tali domande possono essere ottenute sommando i tempi medi di tutte le visite effettuate dalla stessa richiesta presso il centro considerato. Per lo scenario 1FA si ottiene:

$$D_A^{(1FA)} = 0.20 + 0.40 + 0.10 = 0.70 \text{ s}, \quad (10)$$

$$D_B^{(1FA)} = 0.80 \text{ s}, \quad D_P^{(1FA)} = 0.40 \text{ s}. \quad (11)$$

Nello scenario 2FA:

$$D_A^{(2FA)} = 0.20 + 0.40 + 0.15 = 0.75 \text{ s}, \quad (12)$$

$$D_B^{(2FA)} = 0.80 \text{ s}, \quad D_P^{(2FA)} = 0.70 \text{ s}. \quad (13)$$

Assumendo capacità unitaria per ciascun centro, l'utilizzazione nominale del server k è pertanto

$$\rho_k = \lambda D_k. \quad (14)$$

In entrambi gli scenari, la maggiore domanda di servizio è quella del Server B:

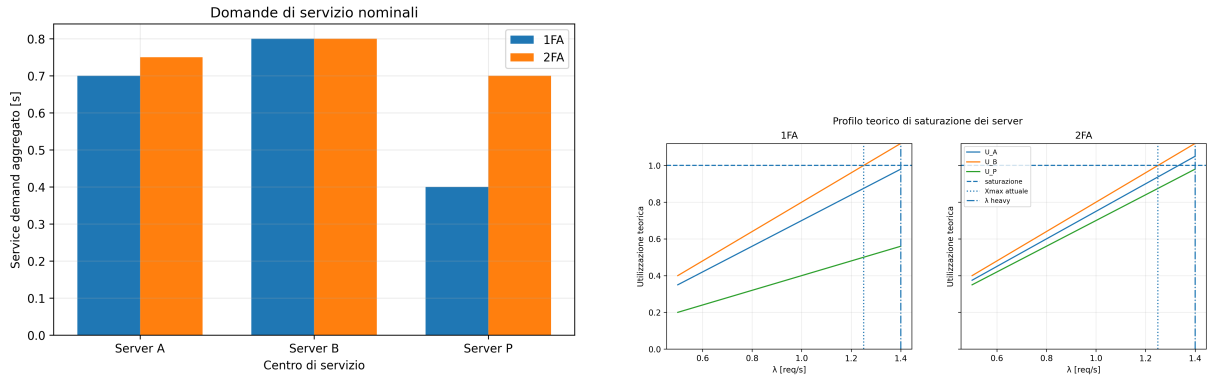
$$D_{\max} = D_B = 0.80 \text{ s}. \quad (15)$$

Il throughput bound nominale della rete è quindi

$$X_{\max} = \frac{1}{D_{\max}} = \frac{1}{0.80} = 1.25 \text{ req/s}. \quad (16)$$

Il Server B rappresenta pertanto il candidato naturale a costituire il collo di bottiglia dell'architettura.

La Figura 2 fornisce una rappresentazione grafica delle domande di servizio aggregate e delle corrispondenti utilizzazioni teoriche al variare del carico.



(a) Domanda di servizio aggregata per centro negli scenari 1FA e 2FA.

(b) Utilizzazioni teoriche dei centri in funzione del tasso di arrivo esterno.

Figura 2: Caratterizzazione analitica del carico offerto. La maggiore domanda di servizio appartiene al Server B ($D_B = 0.80 \text{ s}$) in entrambi gli scenari e determina il throughput bound nominale $X_{\max} = 1.25 \text{ req/s}$.

La Figura 2a evidenzia che l'introduzione della 2FA non modifica la domanda del Server B, mentre incrementa quella del Server A da 0.70 a 0.75 s e soprattutto quella del Server P da 0.40 a 0.70 s. Di conseguenza, come mostrato nella Figura 2b, il Server B conserva la maggiore utilizzazione nominale lungo l'intervallo di carico considerato e raggiunge, per $\lambda = 1.20 \text{ req/s}$,

$$\rho_B = 1.20 \cdot 0.80 = 0.96. \quad (17)$$

Nello scenario 2FA aumentano invece sensibilmente le utilizzazioni del Front-End e del Gateway di Pagamento, pur senza modificare il throughput bound globale, che continua a essere imposto dal Server B.

5.4 Scenario di Heavy Workload

Per valutare la capacità del sistema di sostenere un incremento significativo del traffico viene inoltre considerato uno scenario di **Heavy Workload** con tasso di arrivo

$$\lambda = 1.40 \text{ req/s.} \quad (18)$$

Tale valore rappresenta un incremento di circa il 15% rispetto al massimo carico stazionario esplorato nello scan, pari a $\lambda = 1.20 \text{ req/s}$. Poiché

$$1.40 > X_{\max} = 1.25 \text{ req/s,} \quad (19)$$

la configurazione nominale non soddisfa più la condizione di stabilità del Server B. La sua utilizzazione teorica sarebbe infatti

$$\rho_B = \lambda D_B = 1.40 \cdot 0.80 = 1.12 > 1. \quad (20)$$

Lo scenario Heavy Workload non viene pertanto trattato mediante tecniche di analisi stazionaria. Esso viene studiato separatamente tramite simulazioni a orizzonte temporale finito, con l'obiettivo di osservare la dinamica di accumulo della popolazione presso il collo di bottiglia e il comportamento dei centri a monte e a valle durante il sovraccarico. L'eventuale intervento necessario per rendere sostenibile tale carico viene successivamente analizzato mediante una procedura di *capacity planning*.

6 Modello Computazionale

Il simulatore è stato sviluppato interamente in Java come motore a eventi discreti basato sul paradigma *Next-Event Time Advance*, secondo l'impostazione descritta da Leemis e Park [2]. A differenza di una simulazione *time-driven*, nella quale lo stato viene aggiornato a intervalli temporali prefissati, il clock globale avanza direttamente verso l'istante del prossimo evento rilevante. In questo modo non vengono elaborati intervalli di tempo nei quali lo stato del sistema rimane invariato. Nel modello considerato, gli eventi che possono modificare lo stato della rete sono l'arrivo di una nuova richiesta dall'esterno e i completamenti di servizio presso i Server A, B e P.

6.1 Processor Sharing: Approccio Tag-Based

La simulazione efficiente della disciplina *Processor Sharing* richiede particolare attenzione. In un centro PS contenente n job, ciascun job riceve una frazione $1/n$ della capacità complessiva del server. Ogni nuovo arrivo o completamento modifica quindi istantaneamente la velocità di avanzamento di tutti i job presenti.

Un'implementazione diretta potrebbe mantenere, per ogni job, il servizio residuo in tempo reale e aggiornarlo dopo ogni evento. Tale soluzione richiederebbe di attraversare l'insieme dei job attivi a ogni variazione della popolazione, introducendo un costo $O(n)$ per l'aggiornamento dei tempi residui.

Per evitare tale scansione, il simulatore utilizza un meccanismo basato su **tag di servizio virtuale**. Ogni server k mantiene un proprio orologio virtuale $S_k(t)$, la cui velocità di avanzamento, quando $N_k(t) > 0$, è

$$\frac{dS_k(t)}{dt} = \frac{C_k}{N_k(t)}. \quad (21)$$

Nel modello implementato la capacità è normalizzata a $C_k = 1$. Quando un job entra nel server all'istante in cui l'orologio virtuale vale S_k^{in} , viene campionato il relativo requisito di servizio s e gli viene assegnato il *finish tag*

$$F = S_k^{\text{in}} + s. \quad (22)$$

Il valore F rimane invariato per tutta la permanenza del job nel centro. Il job completa il servizio quando l'orologio virtuale del server raggiunge il suo finish tag. I job attivi sono mantenuti in una **PriorityQueue** ordinata per finish tag. Di conseguenza:

- il job candidato al prossimo completamento è disponibile alla radice della struttura in tempo $O(1)$;
- l'inserimento di un nuovo job nella heap richiede $O(\log n)$;
- l'estrazione del job completato richiede $O(\log n)$;
- l'avanzamento dell'orologio virtuale del server è effettuato in $O(1)$, senza aggiornare individualmente i tempi residui di tutti i job.

Il vantaggio principale dell'approccio consiste quindi nell'evitare l'aggiornamento esplicito di tutti gli n job attivi a ogni evento, aspetto particolarmente importante negli esperimenti in condizioni di sovraccarico, nei quali la popolazione del Server B può diventare molto elevata.

6.2 Esempio Numerico di Funzionamento

Per illustrare il meccanismo, si consideri un Server B con capacità normalizzata $C = 1$ e, per semplicità, si supponga che durante l'intervallo considerato non si verifichino altri eventi oltre a quelli indicati.

- All'istante reale $t = 0$, il server è vuoto e il suo orologio virtuale vale $S = 0$. Entra un job J_1 con requisito di servizio $s_1 = 0.8$ s. Il relativo finish tag è $F_1 = 0 + 0.8 = 0.8$.
- Nell'intervallo $[0, 0.4]$, J_1 è l'unico job presente ($n = 1$), per cui l'orologio virtuale avanza con velocità

$$\frac{dS}{dt} = 1.$$

All'istante $t = 0.4$ risulta pertanto $S = 0.4$.

- All'istante $t = 0.4$ entra un secondo job J_2 . Si supponga che dalla distribuzione di servizio venga campionato un requisito $s_2 = 1.6$ s. Il finish tag assegnato è $F_2 = 0.4 + 1.6 = 2.0$.
- Da questo momento sono presenti due job, quindi la velocità dell'orologio virtuale diventa

$$\frac{dS}{dt} = \frac{1}{2}.$$

Il primo finish tag da raggiungere è quello di J_1 , pari a 0.8.

- La distanza virtuale residua è

$$\Delta S = 0.8 - 0.4 = 0.4.$$

Poiché il tempo virtuale avanza con velocità 0.5, il corrispondente intervallo di tempo reale è

$$\Delta t = \frac{\Delta S}{0.5} = 0.8 \text{ s.}$$

J_1 completa quindi il servizio all'istante reale $t=0.4+0.8=1.2$ s.

- Dopo l'uscita di J_1 rimane soltanto J_2 e la velocità dell'orologio virtuale ritorna a 1. A quel punto $S=0.8$ e il lavoro virtuale ancora necessario per raggiungere il finish tag $F_2 = 2.0$ è $2.0 - 0.8 = 1.2$. Pertanto J_2 completa il servizio dopo ulteriori 1.2 s, all'istante $t=2.4$ s.

Il tempo di permanenza di J_1 è quindi 1.2 s, mentre quello di J_2 è 2.0 s.

6.3 Determinazione del Prossimo Completamento

L'uso dell'orologio virtuale permette di tradurre il finish tag del job alla radice della heap nel corrispondente tempo reale di completamento. Se il server contiene $n > 0$ job, il suo tempo virtuale corrente è S e il finish tag minimo è $F_{\min}$, la distanza virtuale ancora da percorrere è

$$\Delta S = F_{\min} - S. \quad (23)$$

Poiché il tempo virtuale avanza con velocità C/n , il corrispondente tempo reale è

$$\Delta t = \frac{\Delta S}{C/n} = \frac{n(F_{\min} - S)}{C}. \quad (24)$$

Il prossimo completamento del centro viene quindi programmato all'istante

$$t_{\text{dep}} = t + \Delta t. \quad (25)$$

Tale calcolo richiede soltanto la popolazione corrente del server, il suo orologio virtuale e il finish tag minimo, senza attraversare l'intero insieme dei job attivi.

6.4 Risoluzione e Avanzamento del Ciclo di Simulazione

A ogni iterazione il motore confronta l'istante del prossimo arrivo esterno con i prossimi completamenti dei tre centri:

$$t_{\text{next}} = \min(t_{\text{arrival}}, t_{\text{dep},A}, t_{\text{dep},B}, t_{\text{dep},P}).$$

Prima di eseguire l'evento, il simulatore integra le statistiche relative all'intervallo

$$[t, t_{\text{next}}],$$

durante il quale le popolazioni dei centri rimangono costanti. Tali aree vengono utilizzate per il calcolo delle metriche temporali, in particolare popolazione media e utilizzazione. Successivamente gli orologi virtuali dei server vengono fatti avanzare dello stesso intervallo reale

$$\Delta t = t_{\text{next}} - t,$$

tenendo conto della rispettiva popolazione corrente. Infine il clock globale viene posto a t_{next} e viene eseguita la routine associata all'evento. Il Listato 1 mostra la struttura semplificata del passo di simulazione.

Listing 1: Struttura semplificata del passo di simulazione con avanzamento del tempo virtuale.

```
1 // 1. Individuazione del prossimo evento
2 double nextEventTime =
3     Math.min(nextArrival,
4     Math.min(nextDepA,
5     Math.min(nextDepB, nextDepP)));
6
7 double delta = nextEventTime - clock;
8
9
10 // 2. Aggiornamento delle aree statistiche
11 metrics.updateAreas(
12     clock,
13     nextEventTime,
14     serverA.size(),
15     serverB.size(),
16     serverP.size()
17 );
18
19
20 // 3. Avanzamento del tempo virtuale dei server PS
21 serverA.advanceVirtualTime(delta);
22 serverB.advanceVirtualTime(delta);
23 serverP.advanceVirtualTime(delta);
24
25 clock = nextEventTime;
26
27
28 // 4. Esecuzione dell'evento
29 if (clock == nextArrival) {
30
31     handleArrival();
32
33 } else if (clock == nextDepA) {
34
```

```

35     Job completed =
36         serverA.popCompletedJob();
37
38     completed.leaveServer(clock);
39
40     router.routeJob(
41         completed,
42         ServerId.SERVER_A,
43         clock
44     );
45 }
46
47 // Logica analoga per Server B e Server P

```

Nel caso di un arrivo esterno, viene generato un nuovo job di Classe 1, campionato il requisito di servizio della prima visita al Server A e programmato il successivo arrivo. Nel caso di un completamento, il job viene estratto dal relativo server e il **WorkflowRouter** determina la transizione successiva. Se il workflow deve continuare, la classe viene aggiornata, viene campionato il nuovo requisito di servizio e il job viene inserito nel centro successivo. Quando viene completata la terza visita al Server A, il job abbandona invece la rete e viene registrato il relativo tempo di risposta end-to-end.

6.5 Raccolta delle Metriche

Il simulatore mantiene due categorie principali di statistiche.

Metriche associate ai job. Per ogni richiesta completata viene registrato il tempo di risposta end-to-end

$$R_j = t_j^{\text{exit}} - t_j^{\text{arrival}}. \quad (26)$$

Per gli esperimenti di *job tracing* vengono inoltre conservati gli istanti di ingresso e uscita relativi alle cinque visite

$$A_1 \rightarrow B \rightarrow A_2 \rightarrow P \rightarrow A_3.$$

Metriche temporali di stazione. Durante ogni intervallo tra due eventi successivi vengono integrate le popolazioni dei tre centri. Per il generico server k :

$$A_{N_k} = \int N_k(t) dt. \quad (27)$$

Analogamente viene accumulato il tempo durante il quale il server risulta occupato:

$$A_{U_k} = \int \mathbb{I}\{N_k(t) > 0\} dt. \quad (28)$$

Su un intervallo di osservazione di durata T , le corrispondenti stime sono

$$\hat{N}_k = \frac{A_{N_k}}{T}, \quad \hat{U}_k = \frac{A_{U_k}}{T}. \quad (29)$$

Il throughput viene invece ottenuto dal numero di completamenti osservati presso il centro:

$$\hat{X}_k = \frac{C_k(T)}{T}, \quad (30)$$

dove $C_k(T)$ rappresenta il numero di completamenti registrati dal server k durante l'intervallo di misura. Queste grandezze costituiscono la base delle successive fasi di verifica, validazione e analisi sperimentale.

6.6 Criteri di Terminazione

Lo stesso motore supporta due modalità principali di terminazione, utilizzate nelle differenti campagne sperimentali:

- **terminazione per numero di completamenti**, utilizzata nelle simulazioni finalizzate all'analisi a regime e alla raccolta delle osservazioni individuali;
- **terminazione per orizzonte temporale**, utilizzata nell'analisi del transitorio e negli esperimenti in condizioni di sovraccarico.

La separazione tra motore di simulazione e criterio di arresto consente di utilizzare la stessa implementazione del sistema nei diversi protocolli sperimentali descritti nelle sezioni successive.

7 Verifica

La fase di verifica ha l'obiettivo di accertare che l'implementazione del simulatore realizzi correttamente le dinamiche previste dal modello computazionale. A differenza della successiva fase di validazione, nella quale il comportamento dell'intera rete verrà confrontato con il caso di studio e con le previsioni analitiche, la verifica è orientata principalmente all'individuazione di eventuali errori nella logica dell'engine, nella disciplina Processor Sharing e nella generazione delle variabili aleatorie. La procedura è articolata su tre livelli:

1. verifica della coerenza interna delle metriche;
2. verifica controllata dell'implementazione M/G/1-PS;
3. verifica del generatore pseudo-casuale e delle distribuzioni utilizzate dal simulatore.

7.1 Test 1: Coerenza Interna e Legge di Little

Un primo controllo riguarda la coerenza tra quantità statistiche ottenute attraverso meccanismi di raccolta differenti.

Da un lato, il tempo di risposta end-to-end viene misurato direttamente per ciascun job completato:

$$R_j = t_j^{\text{exit}} - t_j^{\text{arrival}}.$$

Dall'altro, la popolazione media dei centri viene ottenuta mediante integrazione continua delle aree di stato.

Per una rete aperta stabile, la Legge di Little applicata all'intero sistema impone

$$E[N_{\text{tot}}] = X E[R], \quad (31)$$

con

$$E[N_{\text{tot}}] = E[N_A] + E[N_B] + E[N_P]. \quad (32)$$

Nel workflow considerato ciascuna richiesta visita il Server B esattamente una volta; il throughput misurato presso B costituisce quindi una stima del throughput delle richieste attraverso la rete. Il controllo viene effettuato sulla configurazione nominale più gravosa all'interno del dominio di stabilità, corrispondente a $\lambda = 1.20$ req/s e scenario 1FA.

Dalla campagna stazionaria definitiva risultano:

$$\begin{aligned} E[R] &= 23.096634 \text{ s}, \\ E[N_A] &= 5.207359, \\ E[N_B] &= 21.548579, \\ E[N_P] &= 0.928583, \\ X_B &= 1.199118 \text{ req/s}. \end{aligned}$$

La popolazione totale ottenuta tramite integrazione delle aree vale pertanto

$$E[N_{\text{tot}}] = 5.207359 + 21.548579 + 0.928583 = 27.684521. \quad (33)$$

Applicando invece la Legge di Little mediante il throughput misurato:

$$X_B E[R] = 1.199118 \cdot 23.096634 = 27.695590. \quad (34)$$

Lo scostamento relativo tra le due determinazioni indipendenti è

$$\epsilon_{\text{Little}} = \frac{|27.695590 - 27.684521|}{27.695590} \cdot 100 \simeq 0.040\%. \quad (35)$$

Lo scostamento risulta quindi ampiamente sub-percentuale. Tale risultato fornisce evidenza della coerenza tra il contatore dei completamenti, la misurazione del tempo di risposta e l'integrazione temporale delle popolazioni.

7.2 Test 2: Verifica Controllata del Processor Sharing M/G/1

La verifica della disciplina Processor Sharing viene effettuata separatamente dalla rete e-commerce completa, in modo da isolare il comportamento del singolo oggetto **PSServer**. A tale scopo è stato costruito l'esperimento **VerificaMG1PS**, nel quale il server opera come una coda M/G/1-PS con $E[S]=1$ s. Sono stati considerati due livelli di carico:

$$\lambda = 0.60 \quad \text{e} \quad \lambda = 0.80,$$

corrispondenti rispettivamente a

$$\rho = 0.60 \quad \text{e} \quad \rho = 0.80.$$

Per una coda M/G/1-PS stabile valgono le relazioni

$$U = \rho = \lambda E[S], \quad (36)$$

$$E[T] = \frac{E[S]}{1 - \rho}, \quad (37)$$

e

$$E[N] = \frac{\rho}{1 - \rho}. \quad (38)$$

Le corrispondenti previsioni teoriche sono quindi:

Tabella 3: Target teorici utilizzati per la verifica controllata del server M/G/1-PS.

λ	ρ	$E[T]$ [s]	$E[N]$
0.60	0.60	2.50	1.50
0.80	0.80	5.00	4.00

Per verificare anche la proprietà di insensibilità rispetto alla forma della distribuzione del servizio, ciascun livello di carico è stato eseguito con due differenti distribuzioni aventi la stessa media:

1. servizio Esponenziale;
2. servizio Iperesponenziale H_2 con $CV = 4$.

Il valore $CV = 4$, superiore a quello $CV = 2$ utilizzato nel modello nominale, è stato scelto deliberatamente per sottoporre l'implementazione del Processor Sharing a una condizione di variabilità particolarmente elevata.

Per ciascuna delle quattro configurazioni sono state utilizzate 64 realizzazioni da 512 completamenti ciascuna e sono stati costruiti intervalli di confidenza al 95%.

Tabella 4: Esito della verifica controllata M/G/1-PS.

Distribuzione	λ	ρ	Target $E[T]$	Compatibilità IC 95%
Esponenziale	0.60	0.60	2.50 s	sì
Iperesponenziale ($CV = 4$)	0.60	0.60	2.50 s	sì
Esponenziale	0.80	0.80	5.00 s	sì
Iperesponenziale ($CV = 4$)	0.80	0.80	5.00 s	sì

In tutte le configurazioni considerate i valori teorici di utilizzazione, popolazione e tempo medio di permanenza risultano compresi nei rispettivi intervalli di confidenza al 95%.

Il risultato è particolarmente significativo nel caso iperesponenziale: nonostante l'incremento della varianza del tempo di servizio, il comportamento medio osservato rimane compatibile con le formule M/G/1-PS. Il test fornisce quindi una verifica diretta dell'algoritmo tag-based implementato dalla classe **PServer**, indipendentemente dalla maggiore complessità introdotta dal routing multi-visita della rete completa.

7.3 Test 3: Verifica del Generatore Pseudo-Casuale

La correttezza del simulatore dipende anche dalla corretta generazione e separazione delle variabili aleatorie utilizzate dai differenti processi stocastici. Il generatore pseudo-casuale utilizzato nel progetto è basato sul generatore di Lehmer fornito da Leemis e Park. A partire da un unico *Master Seed*, la procedura di inizializzazione costruisce diversi stream deterministici, ciascuno assegnato a una specifica sorgente di casualità. Nella configurazione definitiva vengono utilizzati sei stream:

Tabella 5: Assegnazione degli stream pseudo-casuali ai processi stocastici.

Stream	Processo stocastico
0	tempi di interarrivo
1	tempi di servizio del Server A
2	selezione del ramo della distribuzione H_2 del Server B
3	servizio del primo ramo H_2 / servizio Exp di B nel caso di confronto
4	servizio del secondo ramo H_2 del Server B
5	tempi di servizio del Server P

Per il Master Seed utilizzato nelle campagne definitive, **MASTER_SEED**=123456789, gli stati iniziali dei sei stream sono:

Tabella 6: Stati iniziali degli stream RNG derivati dal Master Seed 123456789.

Stream	Stato iniziale
0	123456789
1	2010924726
2	417893401
3	281668658
4	1918141768
5	1524875428

La riproducibilità è stata verificata re-inizializzando il generatore con lo stesso Master Seed e ripetendo la medesima sequenza di chiamate: le sequenze prodotte risultano identiche. È stato inoltre verificato che il consumo di uno stream non modifichi lo stato degli altri stream, condizione necessaria per evitare accoppiamenti artificiali tra arrivi e differenti tempi di servizio.

7.3.1 Verifica delle Distribuzioni di Servizio

Oltre alla riproducibilità delle sequenze uniformi, è stata verificata direttamente la corretta parametrizzazione delle distribuzioni utilizzate dal Server B. Sono stati generati 10^6 campioni con tempo medio target

$$E[S_B] = 0.8 \text{ s.}$$

Per la distribuzione esponenziale, i valori teorici sono

$$E[S_B] = 0.8, \quad \text{Var}(S_B) = 0.8^2 = 0.64, \quad CV = 1.$$

Per la distribuzione iperesponenziale nominale con $CV = 2$:

$$E[S_B] = 0.8, \quad \text{Var}(S_B) = CV^2 E[S_B]^2 = 4 \cdot 0.8^2 = 2.56.$$

I risultati campionari sono riportati nella Tabella 7.

Tabella 7: Verifica empirica delle distribuzioni di servizio del Server B (10^6 campioni, media target 0.8 s).

Distribuzione	Media		Varianza		CV	
	Teoria	Campione	Teoria	Campione	Teoria	Campione
Esponenziale	0.800000	0.800104	0.640000	0.639405	1.000	0.999405
Iperesponenziale	0.800000	0.800484	2.560000	2.570482	2.000	2.002878

Le differenze rispetto ai momenti teorici risultano trascurabili rispetto alla dimensione del campione. La verifica conferma quindi sia la corretta parametrizzazione delle distribuzioni sia il mantenimento della medesima media di servizio nei casi Esponenziale e Iperesponenziale. Nel complesso, i tre livelli di controllo forniscono evidenza della correttezza dei principali componenti sui quali si basa la successiva campagna sperimentale: avanzamento dell'engine, disciplina Processor Sharing, misurazione delle metriche e generazione delle variabili aleatorie.

8 Validazione del Modello

La fase di verifica descritta nella Sez. 7 ha accertato la correttezza dei principali componenti computazionali del simulatore. La validazione considera invece il comportamento della rete completa e verifica che le metriche osservate siano coerenti con il modello analitico e con il caso di studio di riferimento [1].

A tale scopo viene analizzata la risposta del sistema lungo un ampio intervallo di carico, controllando:

- la conservazione dei flussi lungo il workflow multi-visita;
- l'andamento di tempo di risposta e popolazione rispetto alle previsioni analitiche;
- le utilizzazioni dei singoli centri;
- la coerenza con la Legge di Little lungo l'intero scan.

8.1 Protocollo Sperimentale dello Scan di Carico

La validazione viene effettuata mediante la classe `ValidationScan.java`, variando il tasso di arrivo esterno sui 15 punti

$$\lambda \in \{0.50, 0.55, 0.60, \dots, 1.20\} \text{ req/s.} \quad (39)$$

Lo scan viene eseguito per entrambi gli scenari di sicurezza:

- **1FA**, con $D_A = 0.70$ s, $D_B = 0.80$ s e $D_P = 0.40$ s;
- **2FA**, con $D_A = 0.75$ s, $D_B = 0.80$ s e $D_P = 0.70$ s.

Poiché l'obiettivo di questa fase è confrontare il simulatore con il caso di studio originale, il Server B viene configurato con **servizio esponenziale** di media

$$E[S_B] = 0.80 \text{ s,}$$

anziché con la distribuzione iperesponenziale adottata nel modello nominale delle successive campagne sperimentali. Per ciascuna coppia $(\lambda, \text{scenario})$ viene creata una nuova configurazione del sistema e il generatore pseudo-casuale viene reinizializzato dal medesimo Master Seed `MASTER.SEED=123456789`. Ogni run parte da sistema vuoto e termina dopo 200 000 richieste completate. In questa fase non viene applicato alcun warm-up né vengono azzerati i totalizzatori dopo l'avvio. Lo scan di validazione deve infatti essere distinto dalla successiva campagna di stima stazionaria: il suo obiettivo è verificare la corretta risposta strutturale del modello lungo il carico, mentre la rimozione dell'*initialization bias* verrà affrontata nella Sez. 9.

8.2 Conservazione dei Flussi e Throughput

Il workflow deterministico

$$A \rightarrow B \rightarrow A \rightarrow P \rightarrow A$$

impone, per ogni richiesta esterna completata, tre visite al Server A e una visita rispettivamente ai Server B e P.

In assenza di perdite, i throughput attesi sono quindi

$$X_A = 3\lambda, \quad X_B = \lambda, \quad X_P = \lambda. \quad (40)$$

I dati simulati seguono tali relazioni lungo l'intero intervallo di carico. A titolo di esempio, nello scenario 1FA:

$$\begin{aligned}\lambda = 0.50 : \quad X_A &= 1.494652, & X_B &= 0.498221, X_P = 0.498216, \\ \lambda = 1.20 : \quad X_A &= 3.586886, & X_B &= 1.195593, X_P = 1.195587.\end{aligned}$$

I valori teorici corrispondenti sono rispettivamente

$$(1.50, 0.50, 0.50)$$

e

$$(3.60, 1.20, 1.20) \text{ req/s.}$$

Gli scostamenti sono quindi compatibili con la durata finita delle run e non evidenziano perdite o duplicazioni di job durante il *class switching*.

8.3 Validazione delle Curve Globali di Risposta

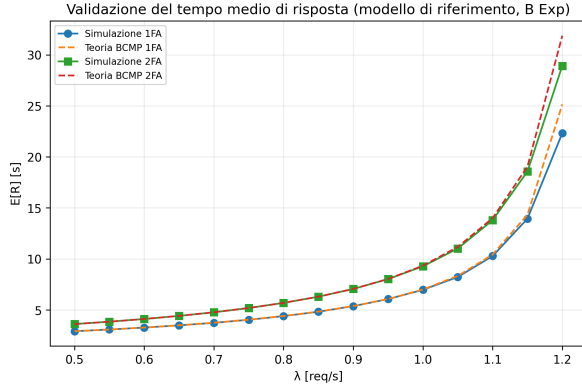
Poiché tutti i centri adottano Processor Sharing, nel dominio stabile le metriche medie possono essere confrontate con le relazioni analitiche della rete BCMP. Il tempo medio di risposta end-to-end è

$$E[R] = \sum_{k \in \{A, B, P\}} \frac{D_k}{1 - \lambda D_k}, \quad (41)$$

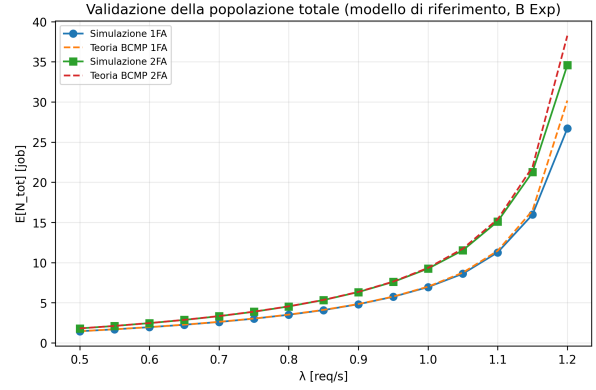
mentre la popolazione media complessiva è

$$E[N_{\text{tot}}] = \sum_{k \in \{A, B, P\}} \frac{\lambda D_k}{1 - \lambda D_k} = \lambda E[R]. \quad (42)$$

Le Figure 3a e 3b confrontano direttamente le previsioni analitiche con i valori ottenuti dal simulatore.



(a) Tempo medio di risposta end-to-end.



(b) Popolazione media totale.

Figura 3: Validazione delle metriche globali lungo lo scan di carico: confronto tra risultati simulati e previsioni analitiche BCMP per gli scenari 1FA e 2FA.

La Tabella 8 riporta quattro punti rappresentativi dello scan.

Tabella 8: Confronto tra teoria BCMP e simulazione in alcuni punti rappresentativi dello scan di validazione.

Scenario	λ	Metrica	Teoria	Simulazione
1FA	0.50	$E[R]$ [s]	2.9103	2.9093
		$E[N_{tot}]$	1.4551	1.4495
	0.80	$E[R]$ [s]	4.4014	4.3920
		$E[N_{tot}]$	3.5211	3.5010
	1.00	$E[R]$ [s]	7.0000	6.9700
		$E[N_{tot}]$	7.0000	6.9451
2FA	1.20	$E[R]$ [s]	25.1442	22.3256
		$E[N_{tot}]$	30.1731	26.6943
	0.50	$E[R]$ [s]	3.6103	3.6136
		$E[N_{tot}]$	1.8051	1.8004
	0.80	$E[R]$ [s]	5.6881	5.6913
		$E[N_{tot}]$	4.5505	4.5368
2FA	1.00	$E[R]$ [s]	9.3333	9.2768
		$E[N_{tot}]$	9.3333	9.2436
	1.20	$E[R]$ [s]	31.8750	28.9226
		$E[N_{tot}]$	38.2500	34.5815

A basso e medio carico l'accordo è molto stretto. Fino a $\lambda = 1.00$ req/s, lo scostamento sul tempo medio di risposta rimane inferiore all'1% in entrambi gli scenari.

L'avvicinamento alla saturazione del Server B determina invece una forte amplificazione dell'effetto delle condizioni iniziali. A $\lambda = 1.20$ req/s, con

$$\rho_B = \lambda D_B = 0.96,$$

la run da 200.000 completamenti conserva ancora un'apprezzabile *initialization bias* dovuto alla partenza da sistema vuoto.

Lo scostamento rispetto alla soluzione stazionaria diventa pertanto -11.21% sul response time nello scenario 1FA e -9.26% nello scenario 2FA. Tale comportamento non viene interpretato come

un errore del simulatore, ma come una motivazione sperimentale per studiare esplicitamente il transitorio e determinare un opportuno periodo di warm-up prima delle successive stime a regime.

8.4 Errore Relativo rispetto alla Soluzione Analitica

Per rendere quantitativamente evidente lo scostamento tra simulazione e soluzione analitica, per ogni punto operativo viene calcolato

$$\epsilon_Y(\lambda) = \frac{\hat{Y}(\lambda) - Y_{th}(\lambda)}{Y_{th}(\lambda)} \cdot 100, \quad (43)$$

dove Y indica la metrica considerata.

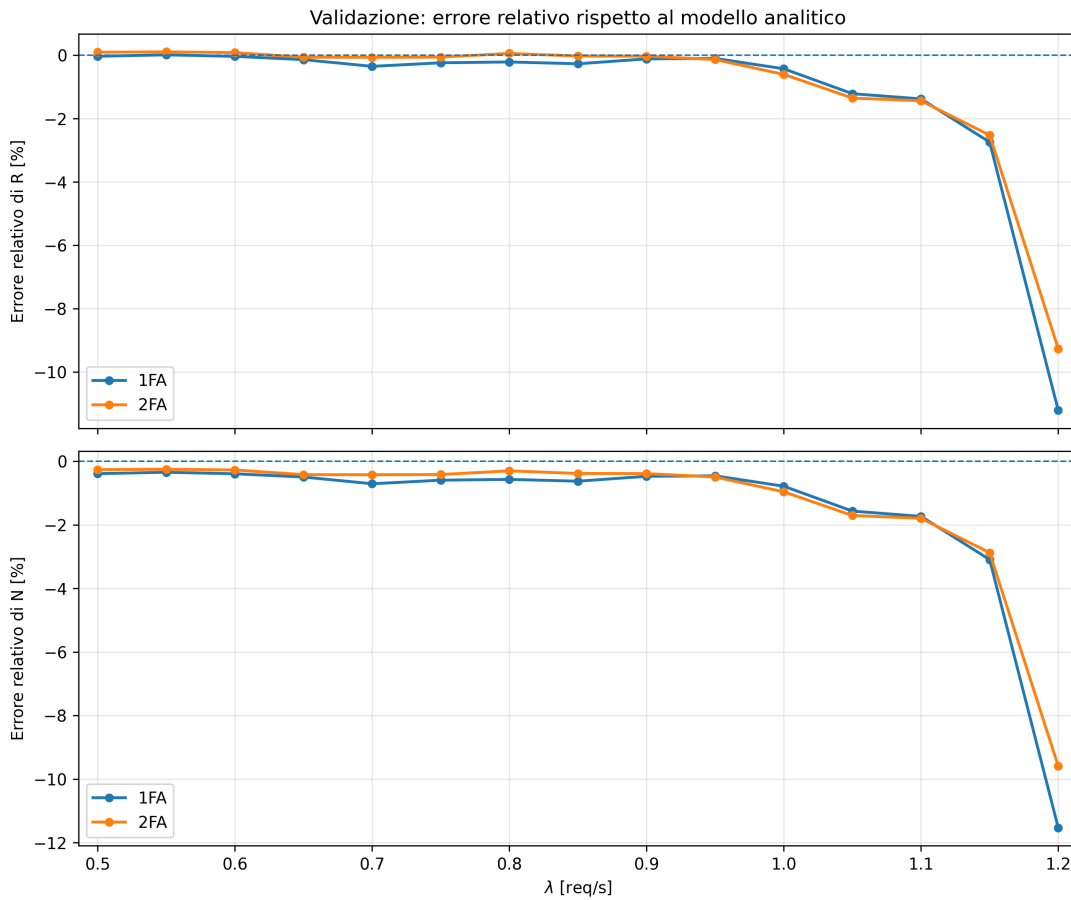


Figura 4: Errore relativo delle stime di tempo di risposta e popolazione rispetto alla soluzione analitica BCMP lungo lo scan di validazione.

La Figura 4 mostra chiaramente due regioni. Per carichi bassi e intermedi l'errore rimane prossimo allo zero, mentre nell'ultimo tratto dello scan aumenta progressivamente per effetto del rallentamento del transitorio in prossimità della saturazione.

La crescita dell'errore a $\lambda = 1.20$ costituisce quindi un'indicazione diretta della necessità di separare la fase di inizializzazione dalla fase di misura nelle campagne stazionarie definitive.

8.5 Validazione delle Metriche di Stazione

Le utilizzazioni teoriche dei centri sono date da $U_k = \lambda D_k$. Diconseguenza, nello scenario 1FA :

$$U_A = 0.70\lambda, \quad U_B = 0.80\lambda, \quad U_P = 0.40\lambda, \quad (44)$$

mentre nello scenario 2FA:

$$U_A = 0.75\lambda, \quad U_B = 0.80\lambda, \quad U_P = 0.70\lambda. \quad (45)$$

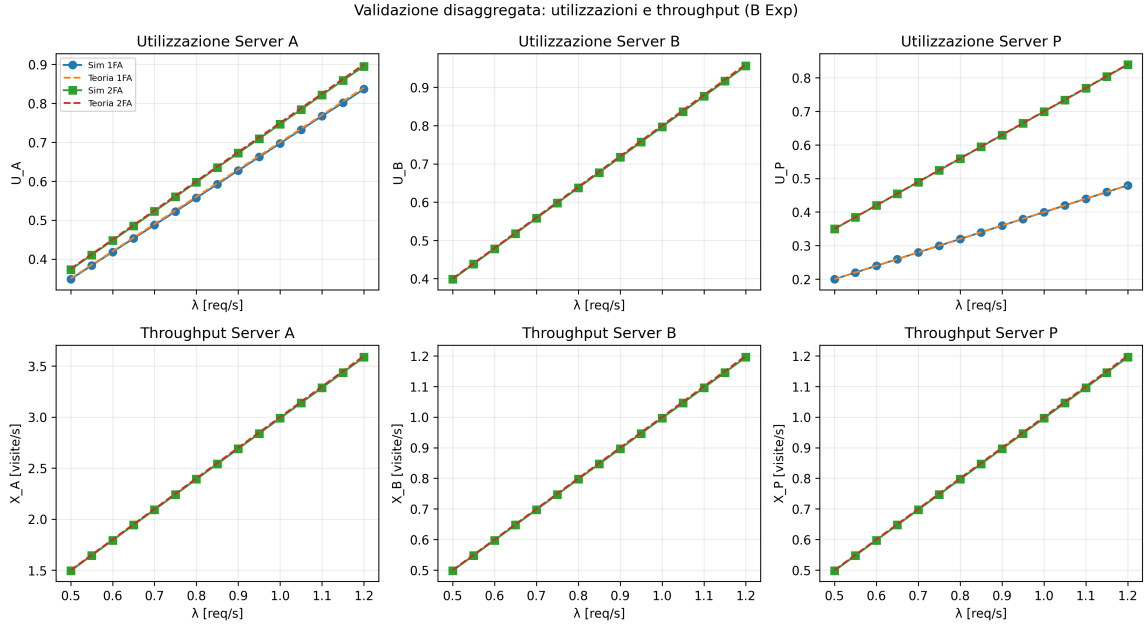


Figura 5: Validazione delle metriche di stazione lungo lo scan di carico. Le utilizzazioni e i throughput simulati vengono confrontati con i rispettivi valori teorici.

Il comportamento osservato conferma la corretta distribuzione del workload:

- **Server B.** La domanda rimane $D_B = 0.80$ s in entrambi gli scenari. Le curve risultano quindi sostanzialmente sovrapposte e raggiungono a $\lambda = 1.20$

$$U_B \simeq 0.956,$$

rispetto al valore teorico 0.960.

- **Server A.** Il passaggio da 1FA a 2FA incrementa la domanda aggregata da 0.70 a 0.75 s. A $\lambda = 1.20$ le utilizzazioni osservate risultano rispettivamente

$$U_A^{(1FA)} = 0.8363$$

e

$$U_A^{(2FA)} = 0.8951,$$

prossime ai valori teorici 0.84 e 0.90.

- **Server P.** L'aumento di D_P da 0.40 a 0.70 s produce la differenza più evidente tra i due scenari. A $\lambda = 1.20$ si osservano

$$U_P^{(1FA)} = 0.4794$$

e

$$U_P^{(2FA)} = 0.8389,$$

in accordo con i valori teorici 0.48 e 0.84.

L'identificazione preliminare del Server B come collo di bottiglia risulta quindi confermata: esso presenta la maggiore utilizzazione in entrambi gli scenari e si avvicina per primo alla saturazione.

8.6 Legge di Little lungo lo Scan

Il controllo puntuale della Legge di Little effettuato nella fase di verifica può essere esteso all'intero scan.

Per ciascuna configurazione vengono confrontate la popolazione totale ottenuta dall'integrazione delle aree di stato e la quantità

$$\lambda E[R]. \quad (46)$$

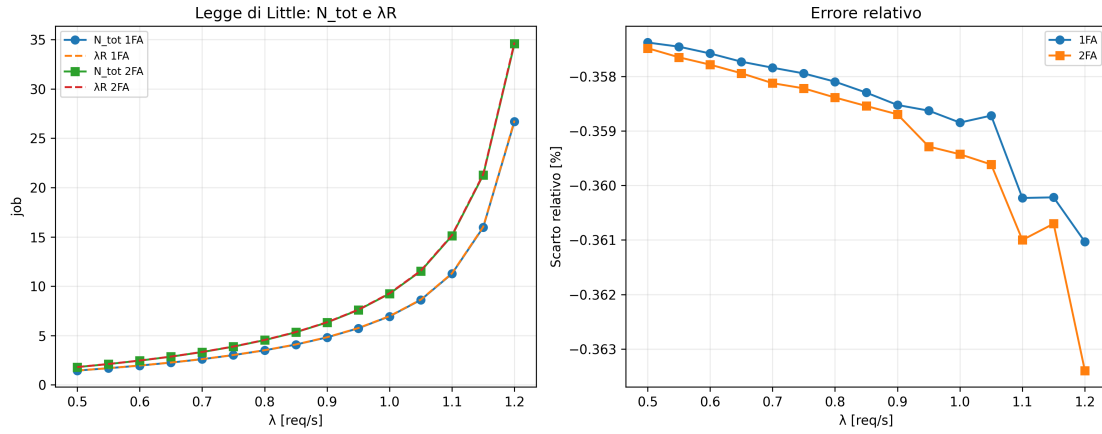


Figura 6: Verifica della Legge di Little lungo lo scan di carico: confronto tra popolazione totale misurata e $\lambda E[R]$, con relativo scostamento percentuale.

La sostanziale sovrapposizione delle due determinazioni lungo l'intero intervallo di carico conferma che, anche nei punti nei quali l'*initialization bias* riduce contemporaneamente le stime di $E[R]$ ed $E[N]$, le due grandezze rimangono internamente coerenti.

8.7 Confronto con il Caso di Studio e Interpretazione del Punto Critico

Nel dominio di basso e medio carico, i risultati simulati riproducono molto bene anche i valori riportati nel caso di studio di riferimento [1].

Le differenze più evidenti emergono nel punto $\lambda = 1.20$ req/s, dove il Server B opera al 96% della propria capacità teorica.

Per lo scenario 2FA, il riferimento riporta $E[R]=34.80$ s e

$$E[N_{\text{tot}}] = 35.90.$$

Tali due valori, se riferiti entrambi allo stesso punto stazionario $\lambda = 1.20$ req/s, non soddisfano simultaneamente la Legge di Little, poiché

$$1.20 \cdot 34.80 = 41.76 \neq 35.90. \quad (47)$$

Viceversa, il valore di popolazione 35.90 implicherebbe

$$R = \frac{35.90}{1.20} \simeq 29.92 \text{ s}. \quad (48)$$

I valori del riferimento devono quindi essere interpretati con cautela nel confronto puntuale al limite della regione operativa.

A ciò si aggiunge la differenza metodologica fondamentale tra lo scan di validazione e le successive stime stazionarie. La procedura qui utilizzata parte da sistema vuoto e raccoglie le statistiche fin dal tempo zero. Quando $\rho_B = 0.96$, il processo di rilassamento è sufficientemente lento da rendere visibile il bias da inizializzazione anche dopo un numero elevato di completamenti. La durata appropriata del warm-up non viene assunta a priori sulla base di questo scostamento, ma viene determinata sperimentalmente nella Sez. 9 mediante più realizzazioni e procedura di Welch. Solo dopo tale calibrazione vengono eseguite le campagne stazionarie definitive presentate nella Sez. 10. Nel complesso, la conservazione dei flussi, la corrispondenza con le relazioni analitiche a basso e medio carico, la corretta evoluzione delle metriche di stazione e il rispetto della Legge di Little forniscono evidenza che il modello implementato riproduce correttamente la struttura prestazionale della rete considerata.

9 Calibrazione Statistica e Analisi del Transitorio

Prima di procedere alla stima delle metriche prestazionali a regime è necessario definire un protocollo sperimentale che riduca l'effetto delle condizioni iniziali e della dipendenza seriale tra osservazioni successive.

La calibrazione viene pertanto articolata in tre fasi:

1. individuazione del punto operativo stabile più prossimo alla saturazione;
2. analisi del transitorio e determinazione del periodo di *warm-up*;

3. analisi dell'autocorrelazione e scelta della dimensione dei batch.

I parametri determinati in questa sezione vengono successivamente utilizzati in modo uniforme nelle campagne stazionarie. La progettazione della campagna sperimentale segue inoltre le linee guida metodologiche per gli studi di simulazione discusse in [3].

9.1 Scelta del Punto Operativo Prossimo alla Saturazione

Come mostrato nella Sez. 5.3, il Server B presenta la maggiore domanda di servizio in entrambi gli scenari:

$$D_{\max} = D_B = 0.80 \text{ s.} \quad (49)$$

Assumendo capacità unitaria, il throughput bound della rete è quindi

$$X_{\max} = \frac{1}{D_{\max}} = \frac{1}{0.80} = 1.25 \text{ req/s.} \quad (50)$$

Per calibrare il simulatore nelle condizioni stabili più gravose tra quelle oggetto dello studio viene selezionato

$$\lambda = 1.20 \text{ req/s.} \quad (51)$$

In tale configurazione il Server B opera con utilizzazione nominale

$$\rho_B = \lambda D_B = 1.20 \cdot 0.80 = 0.96. \quad (52)$$

Il sistema rimane quindi nel dominio di stabilità, ma il collo di bottiglia opera al 96% della propria capacità. Tale punto rappresenta il caso più critico per la determinazione del warm-up e della dimensione dei batch: un protocollo sufficientemente conservativo a $\lambda = 1.20$ può essere riutilizzato anche per i livelli di carico inferiori.

9.2 Analisi del Transitorio e Dimensionamento del Warm-Up

Ogni nuova simulazione parte convenzionalmente da sistema vuoto,

$$N_A(0) = N_B(0) = N_P(0) = 0.$$

Tale condizione non è rappresentativa dello stato stazionario e introduce un *initialization bias*, particolarmente evidente quando il sistema opera in prossimità della saturazione.

Per determinare un periodo di warm-up appropriato vengono analizzati due livelli di carico nello scenario 1FA con il modello nominale del Server B iperesponenziale ($CV = 2$):

$$\lambda = 0.50 \text{ req/s} \quad \text{e} \quad \lambda = 1.20 \text{ req/s.}$$

Il primo rappresenta una configurazione scarsamente congestionata ($\rho_B = 0.40$), mentre il secondo rappresenta il punto stabile più critico ($\rho_B = 0.96$).

Per ciascun valore di λ vengono generate 64 realizzazioni con orizzonte temporale. Ogni nuova configurazione di carico riparte dal medesimo Master Seed, mentre all'interno delle 64 realizzazioni lo stesso oggetto `RandomGenerator` continua a essere utilizzato senza alcun *reseed*. Ogni realizzazione consuma quindi una porzione successiva degli stream pseudo-casuali.

9.2.1 Visualizzazione della Convergenza Multi-Realizzazione

Come primo controllo visuale vengono osservate le statistiche cumulative del tempo di risposta globale R e dell'utilizzazione del Server B U_B .

Le Figure 7 e 8 mostrano cinque delle 64 realizzazioni, insieme alla media d'insieme calcolata su tutte le esecuzioni e al corrispondente valore teorico.

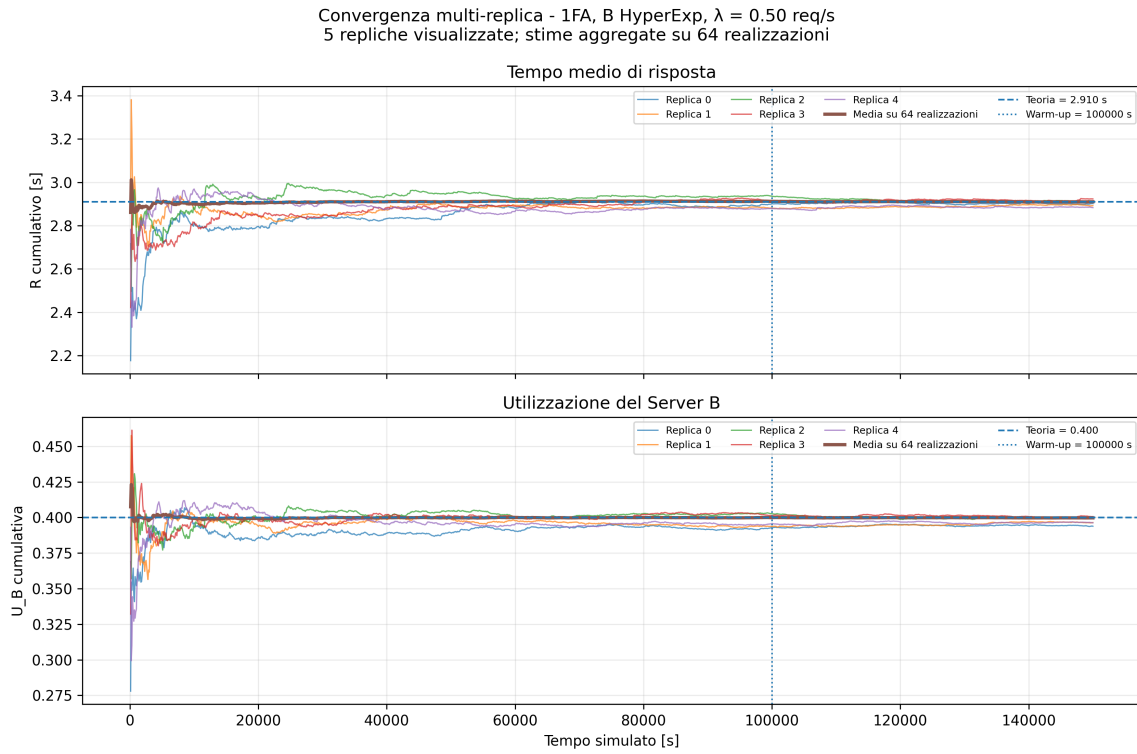


Figura 7: Convergenza multi-realizzazione a basso carico ($\lambda = 0.50$ req/s, 1FA, Server B HyperExp). Sono mostrate cinque realizzazioni individuali e la media sulle 64 realizzazioni per R e U_B . La linea verticale indica il warm-up successivamente adottato.

A basso carico la convergenza è rapida. Al termine dell'orizzonte di 150 000 s, le stime cumulative d'insieme risultano

$$R = 2.91027 \pm 0.00495 \text{ s}, \quad (53)$$

$$U_B = 0.39970 \pm 0.00077, \quad (54)$$

in stretto accordo con i valori teorici

$$R_{th} = 2.91026 \text{ s}, \quad U_{B,th} = 0.40.$$

Convergenza multi-replica - 1FA, B HyperExp, $\lambda = 1.20 \text{ req/s}$
5 repliche visualizzate; stime aggregate su 64 realizzazioni



Figura 8: Convergenza multi-realizzazione al punto operativo critico ($\lambda = 1.20 \text{ req/s}$, 1FA, Server B HyperExp). Sono mostrate cinque realizzazioni individuali e la media sulle 64 realizzazioni per R e U_B .

Il comportamento cambia sensibilmente a $\lambda = 1.20 \text{ req/s}$. L'utilizzazione converge rapidamente verso il valore teorico $U_B = 0.96$, mentre il tempo di risposta presenta una convergenza molto più lenta e una variabilità tra realizzazioni sensibilmente maggiore.

Al termine dell'orizzonte si osserva

$$R = 24.35565 \pm 0.73464 \text{ s}, \quad (55)$$

rispetto al valore teorico

$$R_{th} = 25.14423 \text{ s},$$

mentre

$$U_B = 0.95938 \pm 0.00135 \quad (56)$$

risulta già sostanzialmente coincidente con il valore teorico.

È importante osservare che queste curve sono **cumulative a partire da $t = 0$** : anche quando il sistema ha ormai raggiunto una regione prossima allo stazionario, la media continua a conservare memoria delle osservazioni raccolte durante l'avviamento. Esse costituiscono

quindi un utile controllo visivo della convergenza, ma non vengono utilizzate direttamente per determinare il punto di eliminazione del transitorio.

A titolo di esempio, la Tabella 9 riporta gli stati dei sei stream pseudo-casuali all'inizio delle prime cinque realizzazioni del caso più critico. I valori differenti tra realizzazioni derivano dalla prosecuzione degli stessi stream, non da una nuova inizializzazione manuale.

Tabella 9: Stati degli stream RNG all'inizio delle prime cinque realizzazioni ($\lambda = 1.20$ req/s).

Realizzazione	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5
0	123456789	2010924726	417893401	281668658	1918141768	1524875428
1	38595216	80991370	1531070061	1588875213	1605173710	1281323926
2	946713056	1165778837	607193433	2108733906	759584935	338010215
3	129233818	879275034	459163457	582386220	634138801	1953837861
4	560404076	293218688	1805362616	500590648	688867424	393623146

La determinazione effettiva del warm-up viene effettuata osservando la popolazione del Server B, cioè la stazione che presenta la maggiore domanda di servizio e la dinamica di rilassamento più lenta.

Il modulo `TransientSampler` campiona il sistema ogni

$$\Delta t = 100 \text{ s.} \quad (57)$$

Il valore memorizzato non è la popolazione istantanea all'estremo della finestra, bensì la media temporale della popolazione sull'intervallo:

$$\bar{N}_{B,r}(k) = \frac{1}{\Delta t} \int_{(k-1)\Delta t}^{k\Delta t} N_{B,r}(s) ds, \quad (58)$$

con

$$k = 1, \dots, 1500, \quad r = 1, \dots, 64.$$

Per ogni finestra viene quindi calcolata la media d'insieme sulle 64 realizzazioni:

$$\bar{N}_B(k) = \frac{1}{64} \sum_{r=1}^{64} \bar{N}_{B,r}(k). \quad (59)$$

Per rendere leggibile il trend di fondo, alla sequenza viene applicata una media mobile centrata con semiampiezza $w=150$, corrispondente, nei punti interni della serie, a una finestra complessiva di $2w+1=301$ campioni:

$$\bar{N}_B^{(w)}(k) = \frac{1}{2w+1} \sum_{i=-w}^w \bar{N}_B(k+i). \quad (60)$$

La media mobile viene utilizzata esclusivamente come strumento per l'identificazione visuale del transitorio e non per il calcolo degli intervalli di confidenza delle successive campagne.

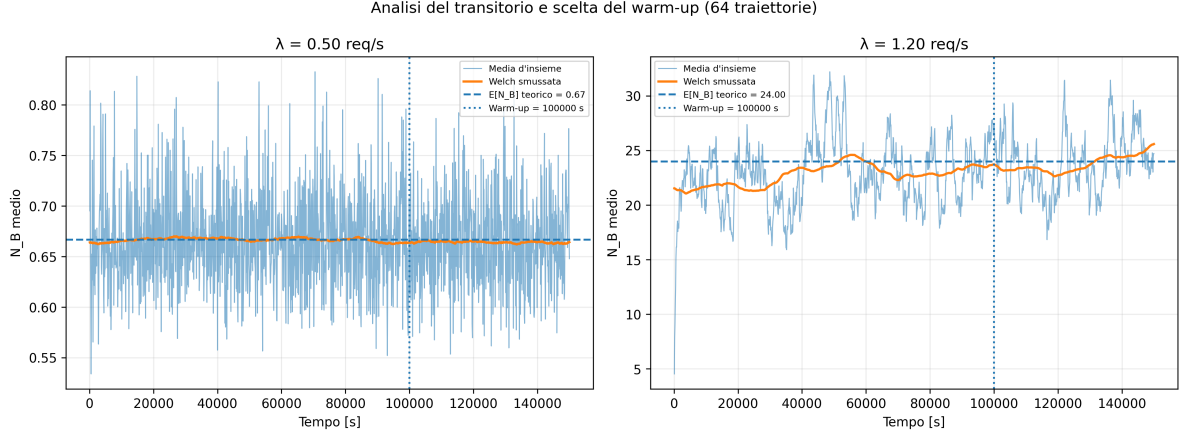


Figura 9: Analisi del transitorio mediante procedura di Welch sulla popolazione del Server B. La media d'insieme su 64 realizzazioni viene confrontata con la versione smussata e con il valore stazionario teorico per $\lambda = 0.50$ e $\lambda = 1.20$ req/s.

Nel caso $\lambda = 0.50$ req/s la curva smussata raggiunge rapidamente il valore teorico

$$E[N_B] = \frac{0.40}{1 - 0.40} = 0.6667,$$

e successivamente oscilla intorno ad esso senza evidenziare una deriva persistente.

Il caso $\lambda = 1.20$ req/s risulta invece nettamente più lento. Il valore stazionario teorico è

$$E[N_B] = \frac{0.96}{1 - 0.96} = 24. \quad (61)$$

La curva smussata si avvicina progressivamente a tale livello nell'ordine delle decine di migliaia di secondi, ma mantiene oscillazioni sensibilmente più ampie rispetto al caso a basso carico.

A partire da circa 60 000 s il trend si colloca nell'intorno del valore teorico; viene tuttavia adottata una soglia più conservativa, comune a tutte le successive configurazioni:

$$T_{\text{warmup}} = \mathbf{100\,000\,s}. \quad (62)$$

La scelta di 100 000 s introduce quindi un margine rispetto alla regione nella quale la curva di Welch inizia ad assestarsi ed è dimensionata sul caso stabile più critico considerato.

Nelle campagne a regime, raggiunto tale istante, le statistiche raccolte durante l'avviamento vengono scartate mediante il reset dei valori, senza reinizializzare il generatore pseudo-casuale. La simulazione prosegue quindi dal medesimo stato del sistema e dalla medesima posizione degli stream RNG.

9.3 Autocorrelazione e Scelta della Dimensione dei Batch

Eliminato il transitorio, rimane il problema della dipendenza seriale tra osservazioni consecutive. In una simulazione di una rete congestionata, due job temporalmente vicini tendono infatti a

osservare condizioni di carico simili e i relativi tempi di risposta non possono essere trattati come osservazioni i.i.d.

La dipendenza viene studiata nella configurazione più critica

$$\lambda = 1.20 \text{ req/s}, \quad 1\text{FA}, \quad B \sim H_2, \quad CV = 2,$$

dopo un warm-up di 100 000 s. Viene analizzata la sequenza di circa $8 \cdot 10^5$ tempi individuali di permanenza presso il Server B contenuta nel file `response_times_B.HyperExp_acf.csv`. La funzione di autocorrelazione campionaria utilizzata è

$$\hat{\rho}_k = \frac{\sum_{i=1}^{N-k} (T_i - \bar{T})(T_{i+k} - \bar{T})}{\sum_{i=1}^N (T_i - \bar{T})^2}. \quad (63)$$

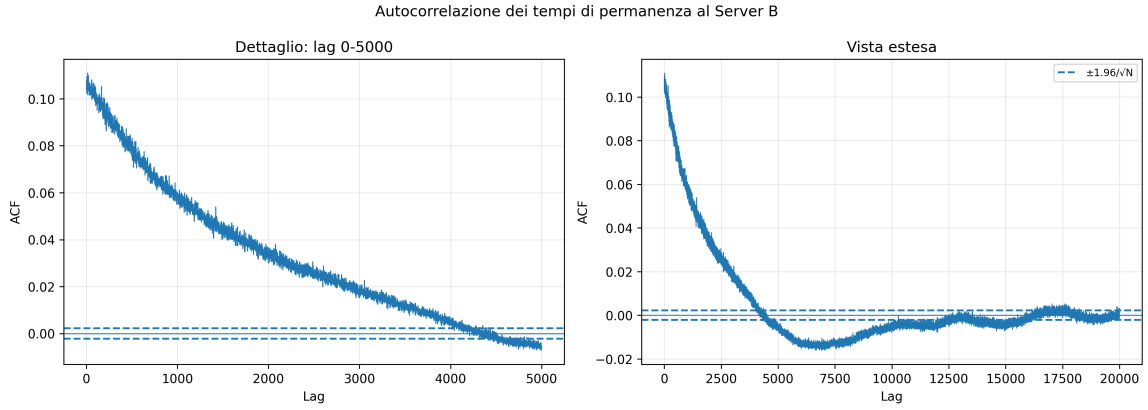


Figura 10: Funzione di autocorrelazione campionaria dei tempi di permanenza al Server B nel punto operativo critico. Le linee tratteggiate rappresentano la banda di riferimento $\pm 1.96/\sqrt{N}$ per un processo non correlato.

La dipendenza seriale è chiaramente visibile. In particolare, la correlazione raggiunge valori prossimi allo zero intorno ai 4300–4400 lag e successivamente oscilla attorno allo zero.

L'analisi della sola ACF delle osservazioni individuali fornisce un primo ordine di grandezza, ma non è sufficiente per scegliere automaticamente la dimensione dei batch. È infatti necessario controllare direttamente la dipendenza residua tra le *batch means*.

Per questo motivo vengono confrontate le dimensioni

$$b \in \{1000, 2000, 4000, 8000, 16000\}.$$

Per ciascun valore di b la sequenza viene suddivisa in blocchi consecutivi e viene calcolata l'autocorrelazione al lag 1 della sequenza delle medie di batch.

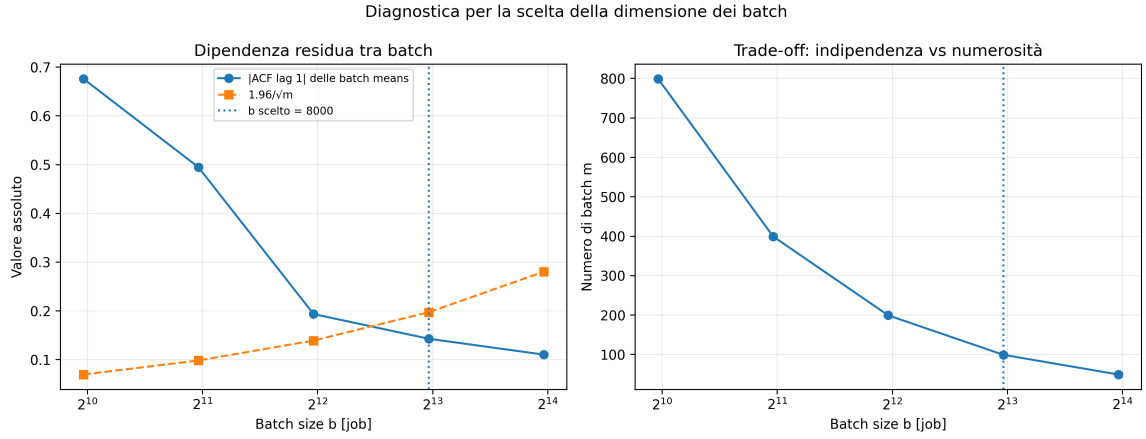


Figura 11: Diagnostica per la scelta della dimensione dei batch. A sinistra è mostrato il valore assoluto dell'ACF al lag 1 delle batch means confrontato con la banda $1.96/\sqrt{m}$; a destra è mostrato il numero di batch disponibili al crescere di b .

Il confronto è particolarmente informativo per le due dimensioni candidate più vicine alla regione di decorrelazione:

$$b = 4000 : \quad |\hat{\rho}_1^{(batch)}| \simeq 0.193 > \frac{1.96}{\sqrt{199}} \simeq 0.139, \quad (64)$$

per cui rimane una dipendenza seriale apprezzabile tra batch consecutivi.

Con

$$b = 8000 : \quad |\hat{\rho}_1^{(batch)}| \simeq 0.142 < \frac{1.96}{\sqrt{99}} \simeq 0.197. \quad (65)$$

La dimensione $b = 8000$ è quindi la prima, tra quelle esaminate in questa regione, per la quale il lag 1 delle batch means rientra nella banda di riferimento. Un ulteriore incremento a $b = 16000$ ridurrebbe ancora la dipendenza, ma diminuirebbe sensibilmente il numero di batch disponibili per la stima della varianza. Nelle successive campagne stazionarie vengono raccolte almeno 800 000 osservazioni valide dopo il warm-up, consentendo di costruire $m=100$ batch per le metriche basate sui completamenti.

La media delle batch means viene utilizzata come stima puntuale, mentre l'intervallo di confidenza al 95% viene costruito mediante la distribuzione t di Student con $m - 1 = 99$ gradi di libertà:

$$\bar{Y} \pm t_{0.975, m-1} \frac{S_Y}{\sqrt{m}}. \quad (66)$$

Per le metriche temporali di stazione, quali popolazione, utilizzazione e throughput, la medesima dimensione nominale viene tradotta in finestre temporali di durata

$$\Delta T = \frac{b}{\lambda}, \quad (67)$$

così da ottenere circa b completamenti esterni per finestra e mantenere coerente il livello di aggregazione tra le differenti metriche.

Il protocollo sperimentale utilizzato nel seguito è quindi definito dai due parametri

$$\boxed{T_{\text{warmup}} = 100\,000 \text{ s}, \quad b = 8000 \text{ job}} \quad (68)$$

che vengono mantenuti costanti nell'intera analisi a regime.

10 Analisi a Regime: Confronto 1FA vs 2FA

Definito il protocollo sperimentale nella Sez. 9, l'analisi si concentra sulla caratterizzazione prestazionale del modello nominale a regime stazionario.

L'obiettivo principale è quantificare il costo prestazionale dell'introduzione della *Two-Factor Authentication* (2FA) rispetto alla configurazione di base 1FA, mantenendo invariato il tasso di arrivo esterno.

La campagna considera l'intero intervallo

$$\lambda \in \{0.50, 0.55, \dots, 1.20\} \text{ req/s}, \quad (69)$$

per un totale di 15 livelli di carico e due scenari di sicurezza.

Il Server B utilizza in questa fase il modello nominale iperesponenziale a due rami con $CV = 2$ e media $E[S_B] = 0.8 \text{ s}$.

10.1 Configurazione dell'Esperimento e Gestione dell'RNG

La campagna viene eseguita mediante la classe `RegimeExperiment.java`.

Per ogni coppia $(\lambda, \text{scenario})$ viene creata una nuova configurazione del sistema e viene inizializzato un nuovo oggetto `RandomGenerator` a partire dal medesimo Master Seed.

La re-inizializzazione viene effettuata esclusivamente quando cambia la configurazione sperimentale. All'interno della stessa configurazione il generatore non viene invece mai reimpostato: la fase di warm-up e la fase di misura consumano in continuità gli stessi stream pseudo-casuali.

Tale politica permette di rendere ogni configurazione deterministica e riproducibile, evitando al tempo stesso di riutilizzare artificialmente la stessa porzione di sequenza durante warm-up e misura.

Per ciascuna configurazione viene applicato il protocollo calibrato nella Sez. 9:

1. **Warm-up.** Il sistema parte vuoto e viene eseguito fino a

$$T_{\text{warmup}} = 100\,000 \text{ s}. \quad (70)$$

2. **Eliminazione del transitorio.** Al termine del warm-up vengono azzerati i totalizzatori statistici mediante `ctx.metrics.reset()`, senza modificare lo stato del sistema né quello del generatore pseudo-casuale.
3. **Misura stazionaria.** La simulazione prosegue fino a raccogliere almeno 800 000 osservazioni valide del tempo di risposta post-warm-up.
4. **Batch Means.** Le osservazioni valide vengono suddivise utilizzando la dimensione $b=8000$, ottenendo 100 batch utilizzati per la costruzione degli intervalli di confidenza al 95%.

I job eventualmente entrati nel sistema prima del termine del warm-up e completati successivamente non vengono utilizzati per il calcolo del tempo di risposta stazionario, poiché la loro storia contiene ancora una porzione del transitorio.

10.2 Batching delle Metriche

Le metriche considerate hanno natura differente e vengono quindi aggregate con due procedure coerenti ma distinte.

Tempo di risposta end-to-end. Il tempo di risposta R è associato alle singole richieste completate. La sequenza dei tempi di risposta validi viene suddivisa direttamente in blocchi consecutivi di $b_R = 8000$ *dallequalisi ottengono 100 medie di batch*.

Metriche temporali di stazione. Popolazione N_i , utilizzazione U_i e throughput X_i vengono invece misurati mediante finestre temporali.

Per mantenere un livello di aggregazione equivalente a circa 8000 richieste esterne per batch, la durata della finestra viene posta pari a

$$\Delta T = \frac{8000}{\lambda}. \quad (71)$$

All'interno di ciascuna finestra vengono integrate le aree di popolazione e di occupazione e vengono conteggiati i completamenti delle stazioni.

Per una generica sequenza di $m = 100$ batch means $\{\bar{Y}_1, \dots, \bar{Y}_m\}$, l'intervallo di confidenza al 95% viene costruito come

$$\bar{Y} \pm t_{0.975, m-1} \frac{S_Y}{\sqrt{m}}. \quad (72)$$

10.3 Tempo di Risposta e Penalità della 2FA

La Figura 12 mostra l'evoluzione del tempo medio di risposta end-to-end al crescere del carico.

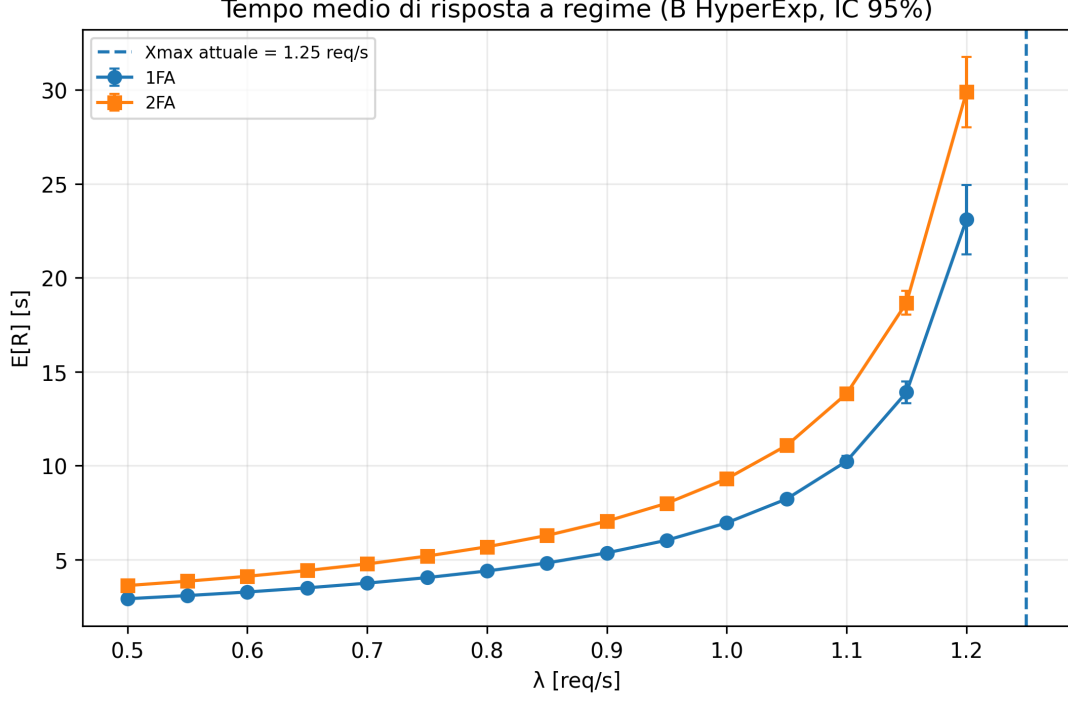


Figura 12: Tempo medio di risposta end-to-end a regime in funzione di λ , con intervalli di confidenza al 95%, per gli scenari 1FA e 2FA.

A basso carico la differenza tra i due scenari è limitata, poiché la componente dominante del response time è costituita principalmente dai tempi di servizio nominali.

All'aumentare di λ , la maggiore domanda introdotta dalla 2FA sui Server A e P viene amplificata dalla congestione. Le due curve si separano quindi progressivamente.

Nel punto più critico dello scan, $\lambda = 1.20$ req/s, si ottiene

$$E[R]_{1FA} = 23.096634 \pm 1.840715 \text{ s}, \quad (73)$$

$$E[R]_{2FA} = 29.890187 \pm 1.881859 \text{ s}. \quad (74)$$

La differenza tra le due stime puntuali è pertanto

$$\Delta R = 29.890187 - 23.096634 = 6.793553 \text{ s}, \quad (75)$$

equivalente a un incremento relativo pari a circa

$$\frac{6.793553}{23.096634} \cdot 100 \simeq 29.41\%. \quad (76)$$

L'impatto incrementale della 2FA lungo l'intero scan è mostrato direttamente nella Figura 13.

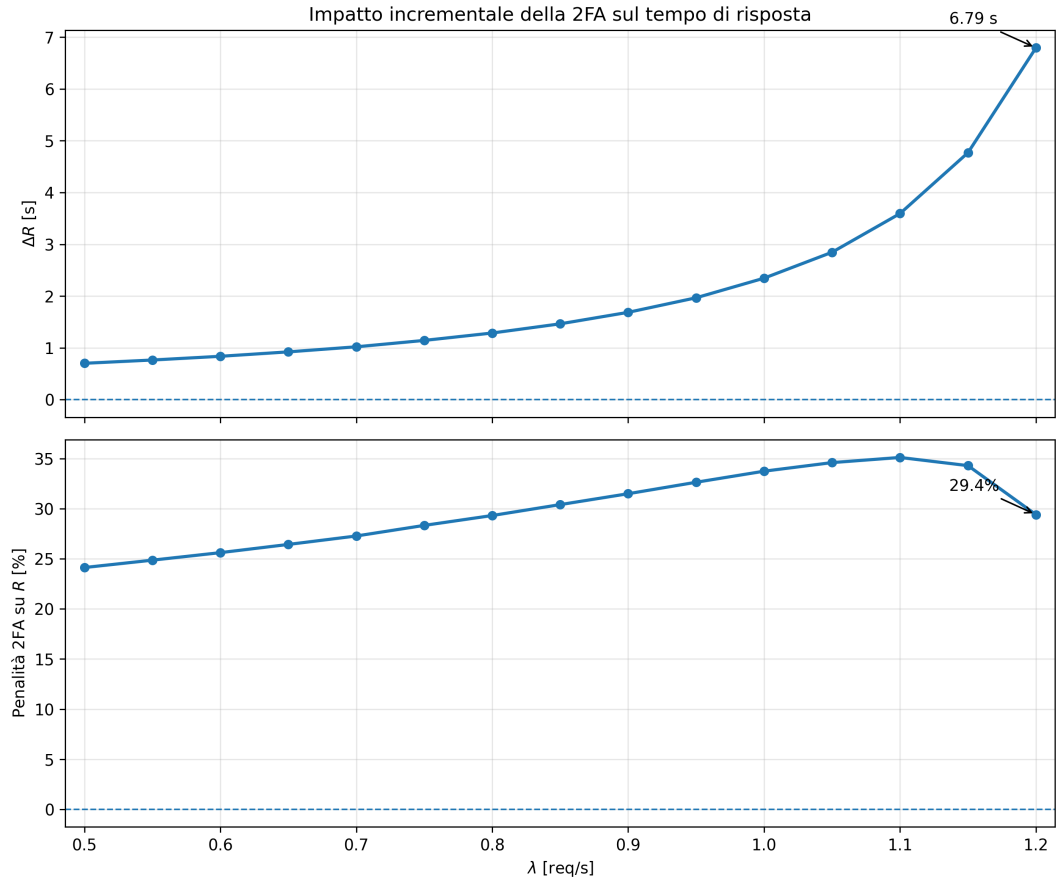


Figura 13: Penalità introdotta dalla 2FA sul tempo medio di risposta end-to-end, espressa in valore assoluto e percentuale rispetto allo scenario 1FA.

10.4 Confronto Quantitativo al Punto Critico

La Tabella 10 riporta le principali metriche ottenute a $\lambda = 1.20$ req/s.

Tabella 10: Confronto delle metriche prestazionali a regime al punto $\lambda = 1.20$ req/s. Le quantità riportate sono media e semi-ampiezza dell'IC al 95%.

Metrica	1FA	2FA	Δ assoluto	Δ %
$E[R]$ [s]	23.0966 ± 1.8407	29.8902 ± 1.8819	+6.7936	+29.41%
$E[N_A]$	5.2074 ± 0.1130	8.8602 ± 0.2955	+3.6528	+70.15%
$E[N_B]$	21.5486 ± 2.2355	21.5833 ± 2.2352	+0.0348	+0.16%
$E[N_P]$	0.9286 ± 0.0063	5.3886 ± 0.1388	+4.4600	+480.30%
U_A	0.8394 ± 0.0023	0.8995 ± 0.0023	+0.0601	+7.16%
U_B	0.9594 ± 0.0040	0.9594 ± 0.0040	≈ 0	< 0.01%
U_P	0.4804 ± 0.0015	0.8407 ± 0.0026	+0.3603	+75.00%
X_A [visite/s]	3.5974 ± 0.0080	3.5975 ± 0.0080	≈ 0	< 0.01%
X_B [visite/s]	1.1991 ± 0.0027	1.1991 ± 0.0027	≈ 0	< 0.01%
X_P [visite/s]	1.1991 ± 0.0027	1.1991 ± 0.0027	≈ 0	< 0.01%

Il confronto evidenzia un punto importante: la 2FA modifica fortemente la congestione sui

Server A e P, ma non modifica il flusso di richieste che attraversa il sistema. Il throughput dei tre centri rimane infatti sostanzialmente identico nei due scenari, mentre cambiano i tempi richiesti per eseguire alcune visite.

10.5 Distribuzione della Congestione tra i Centri

Le Figure 14 e 15 mostrano come la maggiore domanda introdotta dalla 2FA venga redistribuita all'interno della rete.

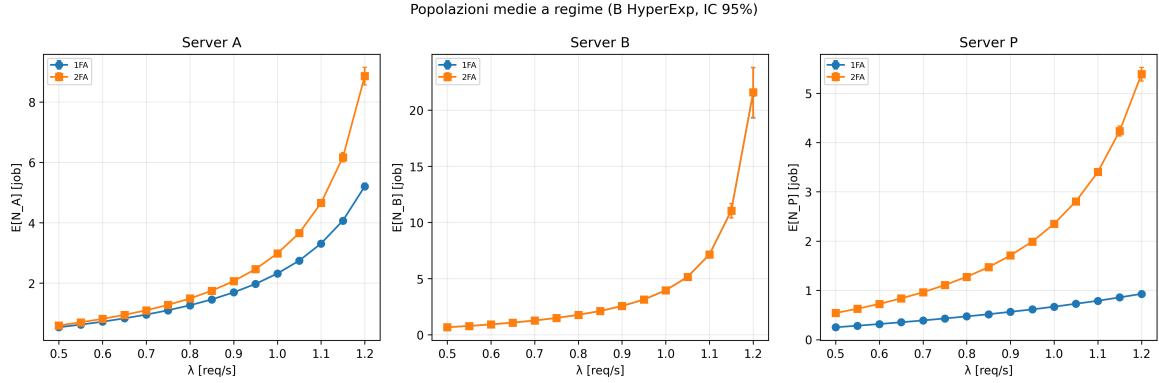


Figura 14: Popolazioni medie dei Server A, B e P a regime al variare del carico per gli scenari 1FA e 2FA.

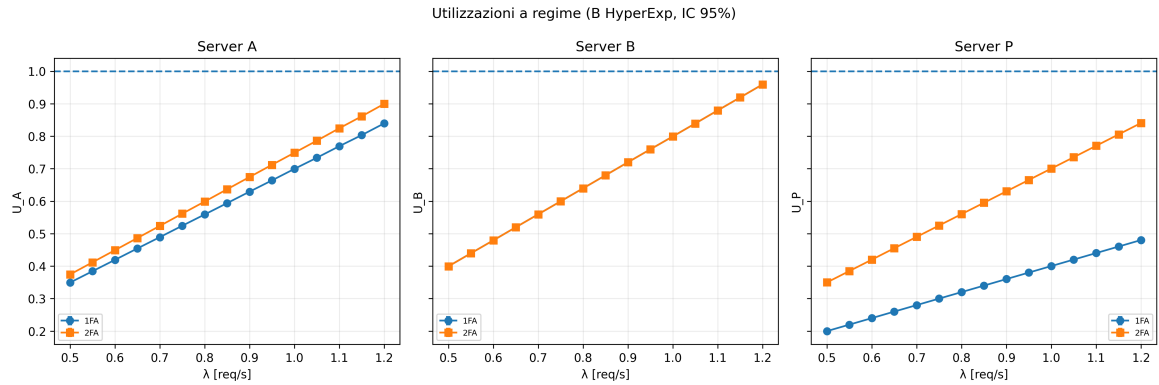


Figura 15: Utilizzazioni medie dei Server A, B e P a regime al variare del carico per gli scenari 1FA e 2FA.

Il comportamento dei tre centri può essere interpretato come segue:

- **Server B – collo di bottiglia invariato.** La 2FA non modifica la domanda del Server B, che rimane pari a $D_B = 0.80$ s. Di conseguenza U_B e N_B risultano sostanzialmente sovrapposti nei due scenari. A $\lambda = 1.20$ si osserva $U_B \simeq 0.9594$, confermando che B rimane il centro più vicino alla saturazione.
- **Server A – maggiore congestione sotto 2FA.** L'incremento della domanda aggregata da 0.70 a 0.75 s porta l'utilizzazione da circa 0.839 a 0.900 al punto di massimo carico. La popolazione media aumenta da circa 5.21 a 8.86 job.

- **Server P – principale effetto della 2FA.** L'aumento della domanda da 0.40 a 0.70 s porta l'utilizzazione da circa 48% a 84%. Per effetto della relazione non lineare tra utilizzazione e congestione, la popolazione media cresce da 0.93 a 5.39 job.

Il Server B continua quindi a imporre il throughput bound globale, ma l'introduzione della 2FA produce un secondo centro fortemente congestionato nel Gateway di Pagamento e aumenta sensibilmente il carico del Front-End.

10.6 Decomposizione del Tempo di Risposta mediante Job Tracing

Per determinare quali visite contribuiscano maggiormente al tempo di risposta complessivo viene eseguito un esperimento dedicato mediante la classe `JobTracingExperiment.java`. La configurazione analizzata è quella nominale più critica:

$$\lambda = 1.20 \text{ req/s}, \quad 1FA, \quad B \sim H_2, \quad CV = 2.$$

Anche in questo caso viene eseguito un warm-up di 100 000 s senza reinizializzare successivamente l' RNG. Il tracing viene abilitato soltanto dopo il warm-up e vengono considerate unicamente le richieste il cui arrivo originario soddisfa

$$t_{\text{arrival}} \geq 100\,000 \text{ s.} \quad (77)$$

La simulazione ha prodotto 808 000 completamenti post-reset, dei quali 807 979 possiedono un tempo di risposta interamente appartenente alla fase di misura. Per l'analisi vengono utilizzate le prime 800 000 richieste valide.

Il tempo medio end-to-end ottenuto dal trace è $E[R]=23.096634$ s, coincidente con la stima utilizzata nella campagna a regime.

La decomposizione media lungo il workflow è riportata nella Tabella 11.

Tabella 11: Decomposizione del tempo medio di risposta sulle cinque visite ($\lambda = 1.20$ req/s, 1FA).

Visita	Tempo medio [s]	Quota di R
A_1	1.2403	5.37%
B	17.9800	77.85%
A_2	2.4822	10.75%
P	0.7744	3.35%
A_3	0.6197	2.68%
Totale	23.0966	100%

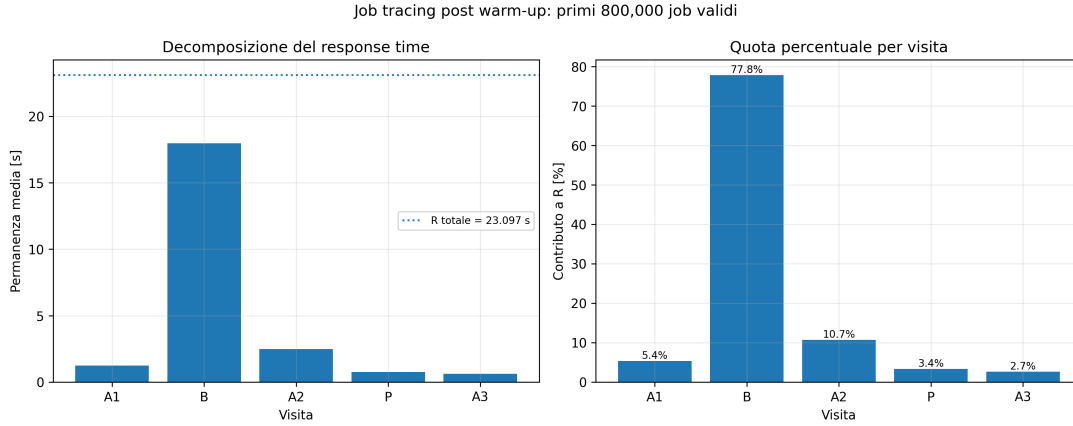


Figura 16: Decomposizione empirica del tempo medio di risposta end-to-end tra le cinque visite del workflow.

Il Server B da solo contribuisce quindi a circa il 77.85% del response time complessivo nel punto critico. Il risultato conferma quantitativamente il ruolo dominante del Database già evidenziato dalle analisi di domanda, utilizzazione e popolazione. Le due visite più onerose al Front-End sono invece A_1 e soprattutto A_2 , mentre il contributo medio del Gateway di Pagamento rimane contenuto nello scenario 1FA. La decomposizione mostra inoltre che la somma dei tempi delle singole visite ricostruisce, entro la precisione numerica, il tempo end-to-end del job, fornendo un ulteriore controllo sulla correttezza del tracciamento.

11 Insensibilità e Variabilità del Servizio al Server B

Una proprietà rilevante delle code M/G/1 sotto disciplina *Processor Sharing* è l'insensibilità di alcune grandezze medie stazionarie rispetto alla forma della distribuzione del tempo di servizio: a parità di tasso di arrivo e tempo medio di servizio, le metriche medie dipendono dalla domanda media ma non dai momenti di ordine superiore della distribuzione.

Tale proprietà non implica tuttavia che l'intera distribuzione dei tempi di permanenza sia insensibile. Varianza, percentili e valori estremi possono cambiare sensibilmente anche quando le metriche medie rimangono compatibili.

L'obiettivo di questa sezione è pertanto duplice:

1. verificare empiricamente se le metriche medie ottenute modificando la distribuzione di servizio del Server B rimangano statisticamente compatibili;
2. misurare l'effetto della maggiore variabilità sulla distribuzione dei tempi individuali di permanenza presso il collo di bottiglia.

11.1 Configurazione Sperimentale

Il confronto viene eseguito mediante la classe `ExpVsHyperExpComparison.java` nel punto stabile più critico:

$$\lambda = 1.20 \text{ req/s}, \quad \text{scenario 1FA.} \quad (78)$$

Vengono considerate due distribuzioni del tempo di servizio del Server B:

1. **Esponenziale**, con $CV=1$;
2. **Iperesponenziale a due rami**, con $CV=2$.

Entrambe hanno la stessa media $E[S_B] = 0.80 \text{ s}$. Per entrambe le configurazioni viene utilizzato lo stesso protocollo di misura. L'impiego dello stesso Master Seed garantisce la riproducibilità delle due configurazioni, ma il confronto tra le distribuzioni iperesponenziali richiede infatti un pattern di consumo degli stream differenterispetto al caso esponenziale.

11.2 Parametrizzazione della Distribuzione Iperesponenziale

La distribuzione H_2 viene parametrizzata in modo da mantenere invariato il tempo medio di servizio

$$E[S_B] = 0.80 \text{ s}$$

e imporre $CV=2$. Indicando con p_1 e p_2 le probabilità dei due rami, la parametrizzazione bilanciata utilizzata nel simulatore è

$$p_1 = \frac{1}{2} \left(1 + \sqrt{\frac{CV^2 - 1}{CV^2 + 1}} \right), \quad p_2 = 1 - p_1. \quad (79)$$

Per $CV^2 = 4$:

$$p_1 \simeq 0.887298, \quad p_2 \simeq 0.112702. \quad (80)$$

Le medie delle distribuzioni esponenziali associate ai due rami sono

$$m_1 = \frac{E[S_B]}{2p_1} \simeq 0.450807 \text{ s}, \quad (81)$$

$$m_2 = \frac{E[S_B]}{2p_2} \simeq 3.549193 \text{ s}. \quad (82)$$

La distribuzione risultante è quindi composta da una grande frazione di richieste con servizio relativamente breve e da una frazione più piccola caratterizzata da richieste molto più onerose.

La generazione pseudo-casuale utilizza stream distinti per le diverse primitive stocastiche:

- lo stream 2 determina il ramo della miscela;
- lo stream 3 genera il valore esponenziale del primo ramo;
- lo stream 4 genera il valore esponenziale del secondo ramo.

Tale separazione evita di accoppiare artificialmente la scelta del ramo con la generazione della relativa durata di servizio.

11.3 Metriche Medie e Proprietà di Insensibilità

La Tabella 12 confronta le principali metriche aggregate ottenute nelle due configurazioni.

Tabella 12: Confronto delle metriche aggregate a $\lambda = 1.20$ req/s, scenario 1FA.

Metrica	Esponenziale	Iperesponenziale	IC 95% sovrapposti
$E[R]$ [s]	24.9064 ± 2.1057	23.0966 ± 1.8407	sì
$E[N_B]$	23.7839 ± 2.4345	21.5486 ± 2.2355	sì
U_B	0.95810 ± 0.00302	0.95944 ± 0.00397	sì
X_B [req/s]	1.19919 ± 0.00265	1.19912 ± 0.00273	sì

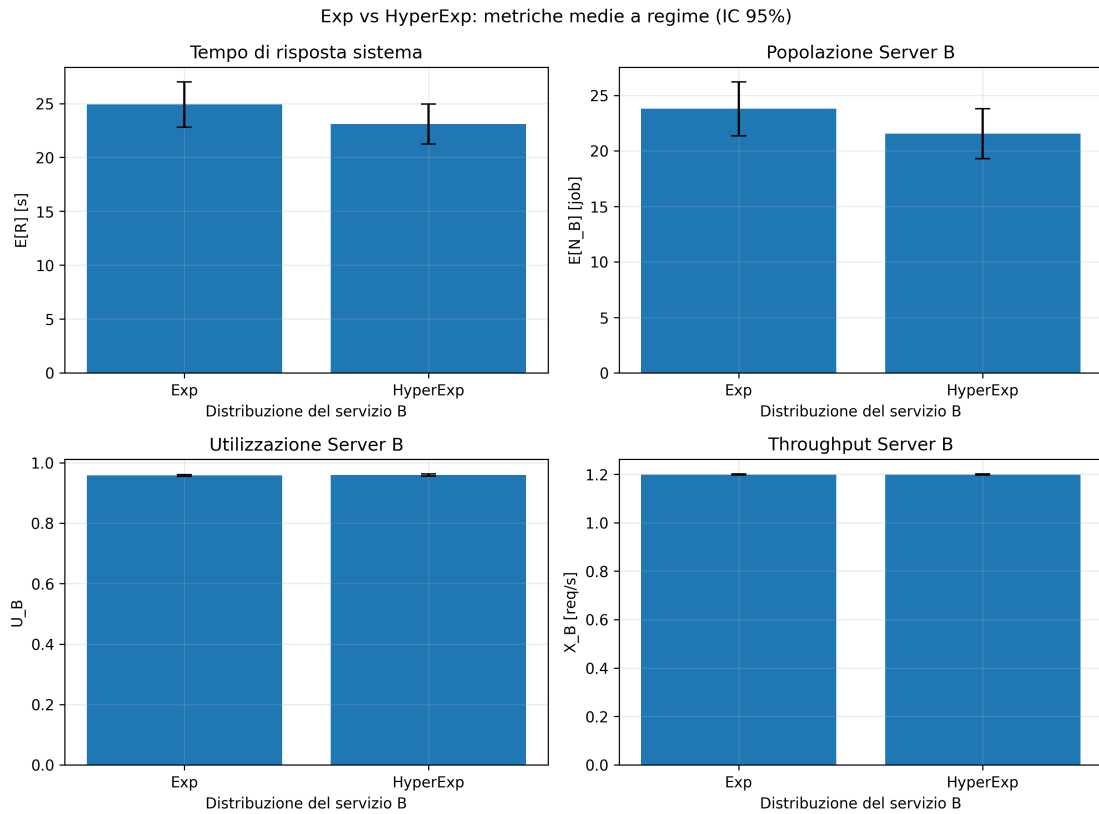


Figura 17: Confronto tra servizio Esponenziale e Iperesponenziale sulle metriche aggregate di sistema e del Server B, con intervalli di confidenza al 95%.

Le stime puntuali non coincidono esattamente, come inevitabile in una simulazione stocastica ad alta utilizzazione, ma i rispettivi intervalli di confidenza al 95% si sovrappongono per tutte le metriche considerate.

In particolare, non emerge evidenza statistica di una differenza del tempo medio end-to-end o della popolazione media del Server B dovuta esclusivamente alla sostituzione della distribuzione esponenziale con quella iperesponenziale a parità di media.

Il risultato è pertanto **coerente** con la proprietà di insensibilità del Processor Sharing. L'esperimento costituisce una verifica empirica e non una dimostrazione del risultato teorico.

11.4 Effetto della Variabilità sui Tempi Individuali

L'assenza di una differenza statisticamente evidente sulle metriche medie non implica che le due distribuzioni producano la stessa Quality of Service.

Per studiare tale aspetto vengono analizzati direttamente i tempi individuali di permanenza al Server B. Nel caso esponenziale sono disponibili 807 996 osservazioni, mentre nel caso iperesponenziale 807 991. Le principali statistiche descrittive ottenute dalle sequenze grezze sono riportate nella Tabella 13.

Tabella 13: Statistiche descrittive dei tempi individuali di permanenza presso il Server B.

Statistica	Exp	HyperExp	Variazione
Media [s]	19.773	18.262	-7.64%
Deviazione standard [s]	34.236	51.180	+49.49%
Coefficiente di variazione	1.731	2.803	+61.86%
P_{50} [s]	7.892	5.253	-33.43%
P_{90} [s]	50.546	39.185	-22.48%
P_{95} [s]	78.915	69.902	-11.42%
P_{99} [s]	165.778	224.663	+35.52%
Massimo [s]	941.121	2402.224	+155.25%

La deviazione standard e il coefficiente di variazione aumentano sensibilmente con il servizio iperesponenziale, mostrando che la dispersione dei tempi individuali è molto maggiore anche quando le metriche aggregate rimangono statisticamente compatibili. La differenza tra le distribuzioni è mostrata più chiaramente osservando direttamente la funzione di distribuzione empirica e la relativa coda.

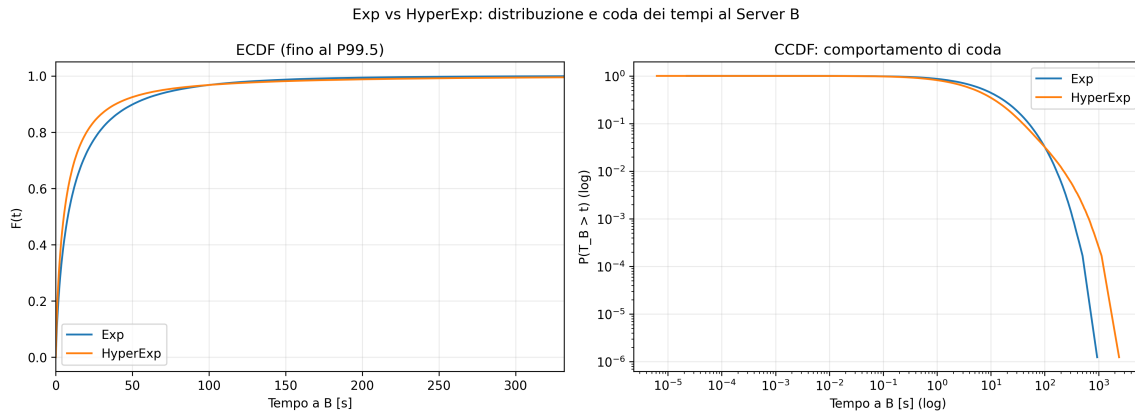


Figura 18: ECDF e CCDF empiriche dei tempi di permanenza al Server B per servizio Esponenziale e Iperesponenziale. La rappresentazione della CCDF rende maggiormente visibile il comportamento della coda.

La distribuzione iperesponenziale presenta un comportamento apparentemente controintuitivo nei quantili centrali. La mediana passa infatti da circa 7.89 s a 5.25 s, e anche P_{90} e P_{95} risultano inferiori rispetto al caso esponenziale. Ciò è conseguenza della struttura della miscela: circa l'88.7% dei requisiti di servizio appartiene al ramo caratterizzato dalla media più piccola.

Sotto Processor Sharing tutti i job attivi ricevono la stessa frazione istantanea di capacità. I job lunghi non impediscono quindi completamente ai job brevi di progredire, come potrebbe

avvenire con una disciplina FIFO per effetto di *head-of-line blocking*. Una parte consistente delle richieste brevi riesce pertanto a completare rapidamente.

Il comportamento cambia invece nella coda estrema. Al percentile P_{99} :

$$P_{99}^{Exp} = 165.78 \text{ s}, \quad (83)$$

mentre

$$P_{99}^{H_2} = 224.66 \text{ s}. \quad (84)$$

L'aumento è pari a circa il 35.5%.

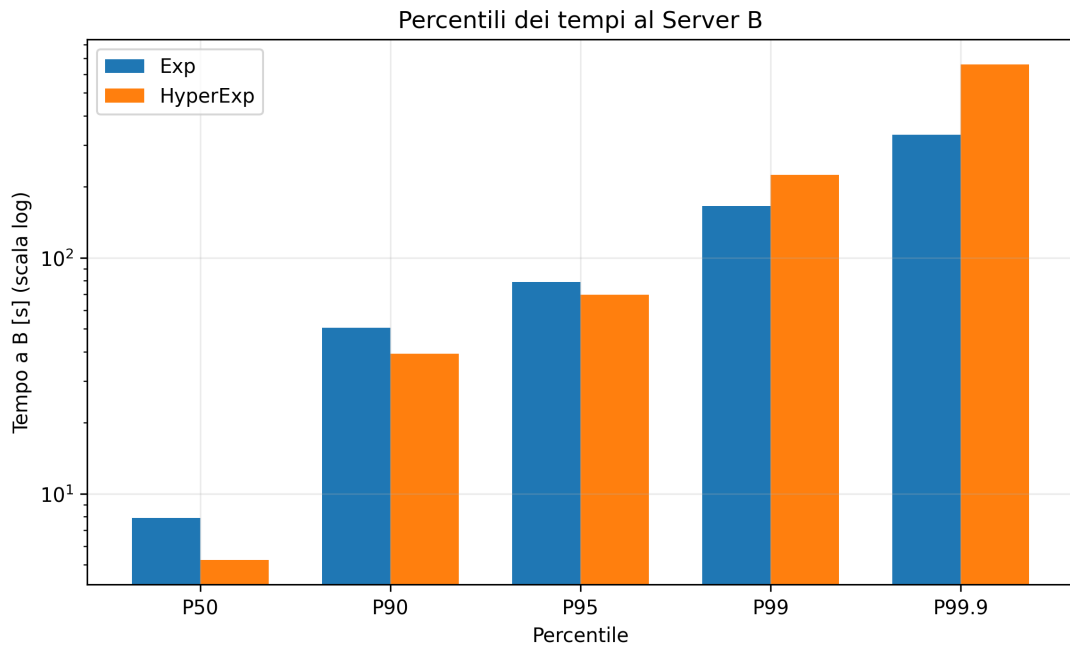


Figura 19: Confronto dei principali percentili dei tempi di permanenza presso il Server B per servizio Esponenziale e Iperesponenziale.

Anche i valori estremi mostrano chiaramente l'effetto della maggiore dispersione. Il massimo osservato passa da circa 941 s nel caso esponenziale a 2402 s nel caso iperesponenziale, corrispondente a circa 40 minuti.

Il massimo campionario non costituisce naturalmente una misura stabile di QoS, ma evidenzia qualitativamente la maggiore capacità della distribuzione iperesponenziale di produrre osservazioni estremamente lunghe.

Nel complesso, l'esperimento mostra quindi due aspetti complementari:

- le metriche medie stazionarie risultano statisticamente compatibili nelle due configurazioni, in accordo con la proprietà di insensibilità del Processor Sharing;
- la distribuzione dei tempi individuali cambia invece significativamente: molti job brevi terminano più rapidamente, mentre la coda estrema diventa più pronunciata.

Di conseguenza, una valutazione prestazionale basata esclusivamente sul tempo medio può risultare insufficiente.

12 Analisi in Condizioni di Sovraccarico (Heavy Load)

Le analisi precedenti hanno caratterizzato il sistema all'interno della regione di stabilità, delimitata dal throughput bound nominale

$$X_{\max} = \frac{1}{D_B} = 1.25 \text{ req/s.} \quad (85)$$

Per valutare la capacità dell'architettura di sostenere un incremento del workload viene considerato il punto

$$\lambda = 1.40 \text{ req/s.} \quad (86)$$

Poiché

$$\rho_B = \lambda D_B = 1.40 \cdot 0.80 = 1.12 > 1, \quad (87)$$

il Server B riceve più lavoro di quanto sia in grado di smaltire.

Il sistema non ammette pertanto una distribuzione stazionaria con popolazione finita e le tecniche utilizzate per la stima a regime, quali warm-up e *Batch Means*, non sono applicabili.

Lo studio viene quindi formulato come esperimento a **orizzonte temporale finito**.

12.1 Configurazione Sperimentale a Orizzonte Finito

Lo scenario nominale viene analizzato mediante la classe `HeavyLoadExperiment.java` secondo il seguente protocollo:

- **Scenario:**

$$\lambda = 1.40 \text{ req/s,} \quad 1\text{FA,} \quad B \sim H_2, \quad CV = 2.$$

- **Orizzonte temporale:**

$$T = 150\,000 \text{ s.}$$

- **Numero di realizzazioni:**

$$r = 50.$$

- **Assenza di warm-up.** L'evoluzione transitoria costituisce l'oggetto stesso dell'esperimento; ogni realizzazione viene pertanto osservata dalla condizione iniziale vuota.

- **Campionamento:** le popolazioni vengono aggregate su finestre temporali di

$$\Delta t = 100 \text{ s.}$$

Per ogni finestra viene calcolata

$$\bar{N}_{i,r}(k) = \frac{1}{\Delta t} \int_{(k-1)\Delta t}^{k\Delta t} N_{i,r}(s) ds. \quad (88)$$

- **Gestione dell’RNG.** Il generatore viene inizializzato dal Master Seed 123456789 all’inizio della configurazione. Le 50 realizzazioni utilizzano in continuità lo stesso oggetto RNG, senza reseed tra una realizzazione e la successiva. Gli stati iniziali degli stream di ogni realizzazione vengono registrati per garantirne la riproducibilità.

Le medie d’insieme e le relative semi-ampiezze degli intervalli di confidenza al 95% vengono calcolate, per ciascuna finestra temporale, sulle 50 realizzazioni.

12.2 Dinamica di Accumulo della Popolazione

La Figura 20 mostra l’evoluzione delle popolazioni medie dei tre centri.

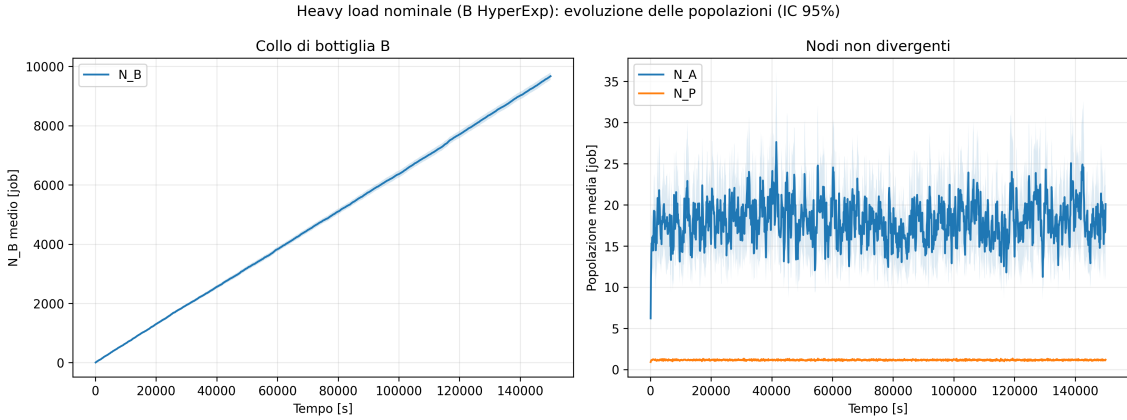


Figura 20: Evoluzione delle popolazioni medie dei Server A, B e P in condizioni di sovraccarico ($\lambda = 1.40$ req/s, 1FA, Server B HyperExp, 50 realizzazioni).

Il comportamento del Server B è qualitativamente differente da quello osservato nel dominio stabile. La sua popolazione cresce per tutto l’orizzonte simulato senza mostrare una tendenza verso un valore di equilibrio.

Nell’ultima finestra, corrispondente a $T = 150\,000$ s, si osserva

$$\bar{N}_B(150\,000) = 9671.27 \pm 122.29 \text{ job.} \quad (89)$$

Tale quantità non deve essere interpretata come una media stazionaria: rappresenta la popolazione media osservata nell’ultima finestra temporale dell’esperimento finito.

Le popolazioni degli altri due centri non mostrano invece una crescita persistente analoga. Nell’ultima finestra si ottiene

$$\bar{N}_A(150\,000) = 20.10 \pm 5.03, \quad (90)$$

$$\bar{N}_P(150\,000) = 1.19 \pm 0.10. \quad (91)$$

Il Server B agisce infatti come un collo di bottiglia che limita il flusso che può proseguire verso le successive fasi del workflow. Il risultato principale dell'esperimento è quindi il confinamento dell'accumulo divergente presso B: gli altri centri presentano fluttuazioni anche ampie, ma non la crescita pressoché lineare osservata sul Database.

12.3 Confronto Esponenziale vs Iperesponenziale in Overload

La proprietà di insensibilità analizzata nella Sez. 11 riguarda metriche stazionarie in condizioni di stabilità. Nel caso $\rho_B > 1$ tale regime non esiste ed è quindi interessante verificare se la distribuzione del servizio influenzi l'evoluzione transitoria della popolazione.

L'esperimento `HeavyExpVsHyperExpComparison.java` confronta:

1. servizio Esponenziale, $CV = 1$;
2. servizio Iperesponenziale H_2 , $CV = 2$;

mantenendo invariati

$$E[S_B] = 0.80 \text{ s}, \quad \lambda = 1.40 \text{ req/s}, \quad T = 150\,000 \text{ s}.$$

Per ciascuna distribuzione vengono eseguite 30 realizzazioni. Ciascuna configurazione riparte dal medesimo Master Seed, mentre le realizzazioni appartenenti alla stessa configurazione continuano a consumare gli stream senza reseed.

L'uso del medesimo Master Seed rende le configurazioni riproducibili, ma il confronto non viene interpretato come un esperimento CRN strettamente appaiato, poiché la generazione iperesponenziale consuma un pattern di stream differente dal caso esponenziale.

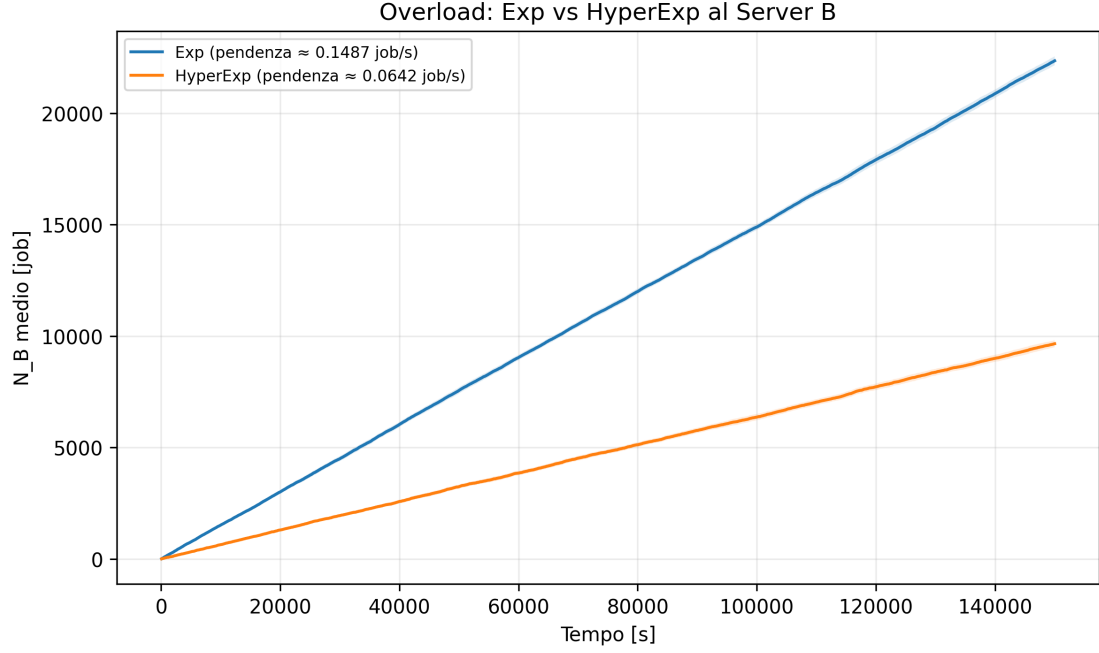


Figura 21: Evoluzione della popolazione del Server B in overload per servizio Esponenziale e Iperesponenziale ($\lambda = 1.40$ req/s, 30 realizzazioni).

Al termine dell'orizzonte si osserva:

Tabella 14: Popolazione media del Server B nell'ultima finestra a $T = 150\,000$ s in condizioni di overload.

Distribuzione	$\bar{N}_B(150\,000)$	Crescita media approssimata
Esponenziale ($CV = 1$)	22353.36 ± 199.31	≈ 0.149 job/s
Iperesponenziale ($CV = 2$)	9659.31 ± 154.04	≈ 0.064 job/s

Nel caso esponenziale, la dinamica osservata è molto vicina alla previsione ottenuta dalla differenza tra il tasso di ingresso e il massimo tasso di completamento:

$$\lambda - X_{\max} = 1.40 - 1.25 = 0.15 \text{ job/s.} \quad (92)$$

Su un orizzonte di 150 000 s, tale drift corrisponde a un accumulo dell'ordine di

$$0.15 \cdot 150\,000 = 22\,500 \text{ job,} \quad (93)$$

coerente con il risultato simulato.

Nel caso iperesponenziale l'accumulo di job nell'orizzonte considerato è invece sensibilmente inferiore, pur essendo il sistema comunque instabile.

La differenza non contraddice la proprietà di insensibilità discussa nella Sez. 11: tale proprietà riguarda il comportamento stazionario di un sistema stabile, mentre qui non esiste una distribuzione stazionaria.

La forma della distribuzione del servizio può quindi influenzare la **traiettoria transitoria a orizzonte finito**. Nel caso H_2 , la forte eterogeneità dei requisiti di servizio produce una diversa composizione della popolazione attiva e un differente numero di completamenti durante l'intervallo osservato.

12.4 Dimensionamento della Capacità

L'instabilità osservata a $\lambda = 1.40$ req/s mostra che il sistema nominale non è in grado di sostenere il workload di picco.

Come primo intervento di capacity planning viene considerato il dimezzamento della domanda del Server B:

$$D_B : 0.80 \longrightarrow 0.40 \text{ s.} \quad (94)$$

La capacità teorica locale di B diventerebbe

$$X'_{\max, B} = \frac{1}{0.40} = 2.50 \text{ req/s,} \quad (95)$$

mentre a $\lambda = 1.40$ la sua utilizzazione scenderebbe a

$$\rho'_B = 1.40 \cdot 0.40 = 0.56. \quad (96)$$

Il potenziamento di B elimina quindi il collo di bottiglia originale, ma la nuova capacità globale dipende dalle domande degli altri centri e risulta diversa nei due scenari di sicurezza.

12.4.1 Scenario 1FA

Dopo l'upgrade:

$$D_A = 0.70, \quad D'_B = 0.40, \quad D_P = 0.40.$$

Il nuovo collo di bottiglia è il Server A e il throughput bound diventa

$$X'_{\max, 1FA} = \frac{1}{0.70} \simeq 1.4286 \text{ req/s.} \quad (97)$$

Poiché $1.40 > 1.4286$, il workload di picco risulterebbe teoricamente sostenibile, ma con

$$\rho_A = 1.40 \cdot 0.70 = 0.98. \quad (98)$$

L'architettura 1FA sarebbe quindi formalmente stabile ma con un margine operativo estremamente ridotto.

12.4.2 Scenario 2FA

Nel caso 2FA, dopo lo stesso potenziamento di B:

$$D_A = 0.75, \quad D'_B = 0.40, \quad D_P = 0.70.$$

Il nuovo collo di bottiglia è ancora il Server A e il throughput bound è

$$X'_{\max,2FA} = \frac{1}{0.75} \simeq 1.3333 \text{ req/s.} \quad (99)$$

Poiché $1.40 > 1.3333$, il solo potenziamento del Server B **non è sufficiente** a rendere stabile lo scenario 2FA.

Le utilizzazioni teoriche al workload di picco sarebbero infatti

$$\rho_A = 1.40 \cdot 0.75 = 1.05, \quad (100)$$

$$\rho_P = 1.40 \cdot 0.70 = 0.98, \quad \rho'_B = 0.56. \quad (101)$$

Il Front-End diventerebbe quindi il nuovo centro instabile, mentre anche il Gateway di Pagamento opererebbe con un margine molto ridotto.

La Figura 22 sintetizza lo spostamento del collo di bottiglia prodotto dall'upgrade del Database.

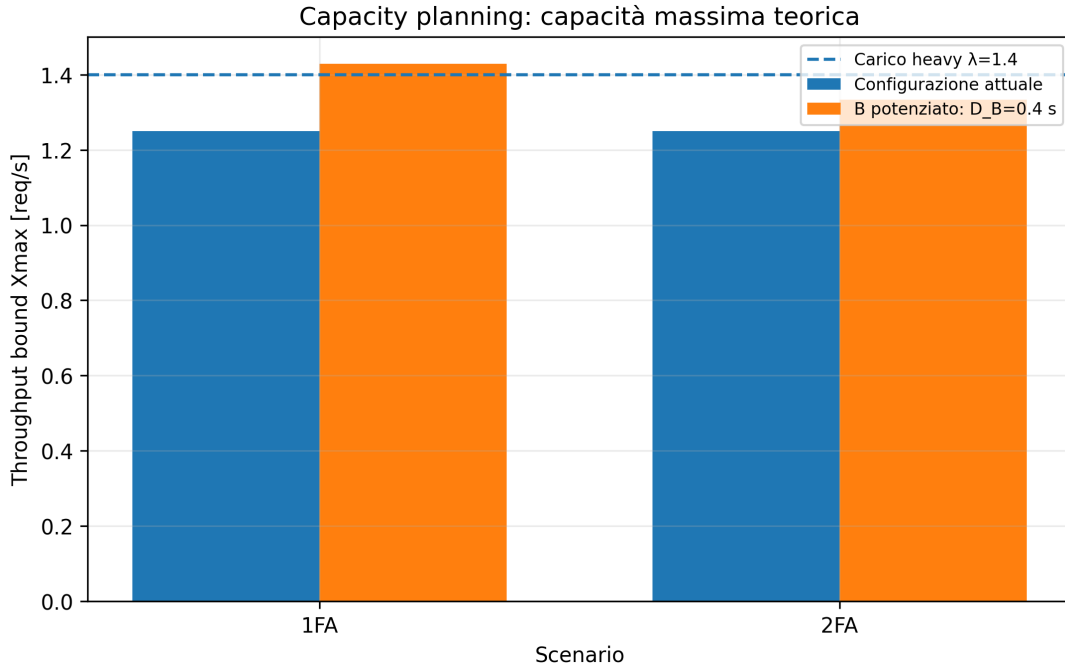


Figura 22: Throughput bound teorico prima e dopo il dimezzamento della domanda del Server B negli scenari 1FA e 2FA.

La Figura 23 mostra invece le utilizzazioni che i singoli centri assumerebbero al workload di picco dopo l'intervento.

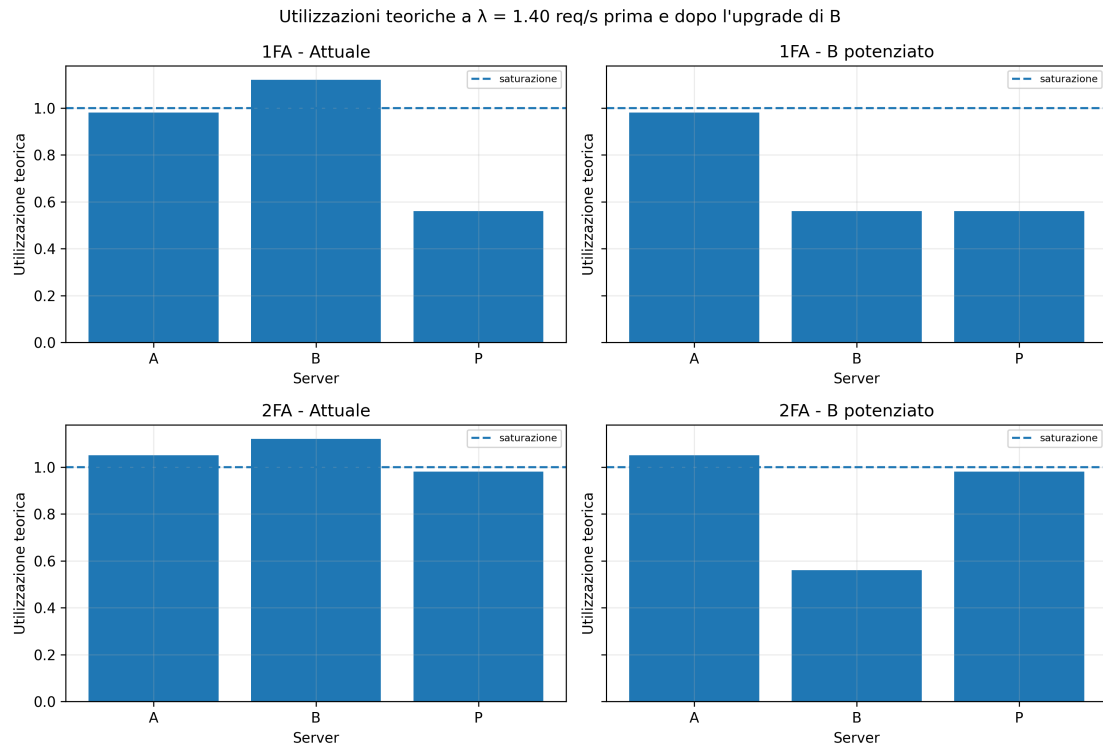


Figura 23: Utilizzazioni teoriche dei centri a $\lambda = 1.40$ req/s dopo il potenziamento del Server B.

L'analisi conduce quindi a una conclusione differente nei due scenari:

- in **1FA**, il potenziamento di B è teoricamente sufficiente a ristabilire la stabilità, ma lascia il Server A al 98% di utilizzazione;
- in **2FA**, il medesimo intervento non è sufficiente: il Server A raggiungerebbe $\rho_A = 1.05$ e richiederebbe a sua volta un incremento di capacità.

Di conseguenza, un dimensionamento orientato a sostenere in modo robusto $\lambda = 1.40$ req/s, soprattutto nello scenario 2FA, deve considerare non soltanto il Database ma anche il Front-End e il ridotto margine residuo del Gateway di Pagamento.

13 Conclusioni

Il lavoro ha sviluppato un simulatore a eventi discreti in Java per lo studio di un workflow e-commerce multi-visita

$$A \rightarrow B \rightarrow A \rightarrow P \rightarrow A$$

modellato come rete di code aperta multiclasse sotto disciplina *Processor Sharing*.

L'implementazione utilizza il paradigma *Next-Event Time Advance* e un meccanismo basato su tag di servizio virtuale per rappresentare efficientemente la condivisione della capacità tra i job attivi.

La fase di verifica ha controllato separatamente la coerenza interna delle metriche, l'implementazione della disciplina M/G/1-PS e il generatore pseudo-casuale. Nel controllo mediante Legge di Little al punto più critico della campagna stazionaria, le due determinazioni indipendenti della popolazione totale differiscono di circa lo 0.04%.

La validazione della rete completa ha inoltre mostrato la corretta conservazione dei flussi e una stretta corrispondenza con le previsioni analitiche a basso e medio carico. Lo scostamento osservato nello scan cold-start in prossimità di $\rho_B = 0.96$ ha motivato l'analisi esplicita del transitorio e la successiva eliminazione dell'*initialization bias*.

La calibrazione sperimentale, condotta mediante 64 realizzazioni, procedura di Welch e analisi dell'autocorrelazione, ha portato alla scelta

$$T_{\text{warmup}} = 100\,000 \text{ s}, \quad b = 8000 \text{ job.} \quad (102)$$

Tali parametri sono stati quindi utilizzati nelle campagne stazionarie definitive.

Prestazioni e collo di bottiglia. L'analisi conferma che il **Server B** costituisce il collo di bottiglia nominale dell'architettura in entrambi gli scenari di sicurezza. La sua domanda $D_B = 0.80 \text{ s}$ impone il throughput bound $X_{\max} = 1.25 \text{ req/s}$.

A $\lambda = 1.20 \text{ req/s}$ l'utilizzazione simulata raggiunge circa

$$U_B = 0.959,$$

sia in 1FA sia in 2FA.

Il job tracing mostra inoltre che, nello scenario 1FA al punto critico, la permanenza presso B contribuisce per circa il 77.85% al tempo di risposta end-to-end.

Impatto della 2FA. L'introduzione della 2FA non modifica il carico del Server B, ma incrementa quello del Front-End e soprattutto del Gateway di Pagamento.

A $\lambda = 1.20 \text{ req/s}$, il tempo medio di risposta passa da

$$23.0966 \pm 1.8407 \text{ s}$$

in 1FA a

$$29.8902 \pm 1.8819 \text{ s}$$

in 2FA, con un incremento della stima puntuale di circa 6.79 s (+29.4%).

La popolazione media del Gateway P cresce da circa 0.93 a 5.39 job, mentre la sua utilizzazione passa dal 48% all'84%.

La maggiore sicurezza introduce quindi un costo prestazionale significativo, pur lasciando invariato il collo di bottiglia principale nel dominio di carico studiato.

Variabilità del servizio. Il confronto tra servizio Esponenziale e Iperesponenziale sul Server B non evidenzia una differenza statisticamente significativa nelle metriche medie stazionarie considerate, risultato coerente con la proprietà di insensibilità del Processor Sharing.

La distribuzione dei tempi individuali cambia tuttavia in maniera sostanziale. Con servizio iperesponenziale la deviazione standard del tempo di permanenza a B cresce da circa 34.24 a 51.18 s e il coefficiente di variazione da 1.73 a 2.80.

Anche la coda estrema risulta più pronunciata: il percentile P_{99} passa da circa 165.8 a 224.7 s, mentre il massimo osservato cresce da circa 941 a 2402 s.

Le metriche medie non sono quindi sufficienti, da sole, a caratterizzare completamente la Quality of Service del sistema.

Comportamento in sovraccarico. Con

$$\lambda = 1.40 \text{ req/s},$$

il Server B presenta

$$\rho_B = 1.12 > 1$$

e la rete nominale non ammette un regime stazionario. Nell'esperimento a orizzonte finito con servizio iperesponenziale, la popolazione media del Server B nell'ultima finestra raggiunge circa 9671 job dopo 150 000 s e continua a crescere.

Nel confronto dedicato, il caso Esponenziale raggiunge circa 22353 job contro circa 9659 del caso Iperesponenziale. Tale differenza descrive una dinamica transitoria a orizzonte finito e non contraddice l'insensibilità stazionaria, che non è applicabile nel dominio instabile.

Il sistema nominale non è quindi in grado di sostenere $\lambda = 1.40 \text{ req/s}$.

Capacity planning. Il dimezzamento della domanda del Server B,

$$D_B : 0.80 \rightarrow 0.40 \text{ s},$$

rimuove B come collo di bottiglia ma produce risultati differenti nei due scenari di sicurezza.

In 1FA, il nuovo limite è imposto dal Server A:

$$X'_{\max,1FA} = \frac{1}{0.70} \simeq 1.4286 \text{ req/s}.$$

Il workload $\lambda = 1.40$ sarebbe quindi teoricamente sostenibile, ma con $U_A \simeq 98\%$ e quindi con un margine operativo minimo.

In 2FA, invece,

$$X'_{\max,2FA} = \frac{1}{0.75} \simeq 1.3333 \text{ req/s},$$

per cui il medesimo upgrade del Database non è sufficiente: a $\lambda = 1.40$ il Server A raggiungerebbe

$$\rho_A = 1.05.$$

Un dimensionamento destinato a sostenere il workload di picco anche con 2FA deve quindi intervenire almeno anche sulla capacità del Front-End; inoltre, con $D_P = 0.70$ s, il Gateway di Pagamento opererebbe già al 98% della propria capacità e offrirebbe un margine molto ridotto.

Considerazioni finali. Nel complesso, il progetto mostra come una rete di code aperta con *class switching* permetta di rappresentare un workflow deterministico multi-visita e di analizzarne sia il comportamento nominale sia le condizioni prossime e superiori alla saturazione.

La combinazione di verifica, validazione, analisi del transitorio, Batch Means, studio della variabilità e simulazioni a orizzonte finito ha permesso di rispondere agli obiettivi iniziali e di individuare sia i limiti prestazionali dell'architettura attuale sia le principali direzioni di capacity planning.

Il progetto è stato svolto individualmente; non è stato pertanto introdotto un modello migliorativo strutturale obbligatorio. L'analisi di capacity planning costituisce una valutazione *what-if* delle possibili evoluzioni dell'architettura e non una nuova configurazione simulata.

Riferimenti bibliografici

- [1] G. Serazzi, *Performance Engineering*, Springer Open Access.
- [2] L. M. Leemis, S. K. Park, *Discrete-Event Simulation: A First Course*, Pearson Prentice Hall, 2006.
- [3] S. Kurkowski, T. Camp, M. Colagrosso, "MANET Simulation Studies: The Incredibles," *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 9, no. 4, 2005.